

Python Programming & IT Core Concepts

Complete Interview Preparation Guide

From Scratch to Associate Software Engineer Level

L1 & L2 Interview Preparation

Topics Covered
Python Fundamentals to Advanced
Object-Oriented Programming
Data Structures & Algorithms
Database & SQL Basics
Operating Systems & Networking
Web Technologies & APIs
Version Control & SDLC
Interview Questions with Answers

Table of Contents

PART 1: PYTHON PROGRAMMING

1. Introduction to Python	4
2. Python Installation & Setup	5
3. Variables, Data Types & Operators	6
4. Strings in Python	10
5. Control Flow Statements	13
6. Functions	16
7. Data Structures: Lists, Tuples, Sets, Dicts	20
8. File Handling	26
9. Exception Handling	28
10. Object-Oriented Programming	30
11. Modules and Packages	36
12. Iterators and Generators	38
13. Decorators	40
14. Lambda, Map, Filter, Reduce	42
15. Comprehensions	44
16. Regular Expressions	46
17. Multithreading & Multiprocessing	48
18. Python Standard Library	50

PART 2: IT CORE CONCEPTS

19. Data Structures	52
20. Algorithms & Complexity	58
21. Database & SQL	64
22. Operating Systems	70
23. Computer Networks	74
24. Software Development Lifecycle	78
25. Version Control (Git)	80
26. Web Technologies & REST APIs	82
27. Cloud Computing Basics	85
28. Software Testing	87

PART 3: INTERVIEW PREPARATION

29. L1 Coding Interview Questions	89
30. L2 Coding Interview Questions	95

31. Theory & Conceptual Questions	100
32. HR & Behavioral Questions	104
33. Tips & Best Practices	106

PART 1: PYTHON PROGRAMMING

This section covers Python from the absolute basics to advanced concepts required for an Associate Software Engineer role.

1. Introduction to Python

What is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability with its use of significant indentation and a clean, English-like syntax.

Key Features of Python

- **Easy to learn:** Simple syntax similar to English.
- **Interpreted:** Code runs line-by-line, no compilation needed.
- **Dynamically typed:** No need to declare variable types.
- **Object-oriented:** Supports classes, objects, and inheritance.
- **Cross-platform:** Works on Windows, macOS, Linux.
- **Free and open-source:** Anyone can use and modify it.
- **Huge standard library:** Built-in modules for many tasks.
- **Large community:** Vast third-party packages via PyPI.

Where is Python Used?

- Web Development (Django, Flask, FastAPI)
- Data Science & Machine Learning (NumPy, Pandas, scikit-learn, TensorFlow)
- Automation & Scripting
- Game Development (Pygame)
- Desktop Applications (Tkinter, PyQt)
- Scientific Computing
- DevOps and Cloud Automation

Python 2 vs Python 3

Python 3 is the current and only supported version. Python 2 reached end-of-life in January 2020. Always use Python 3.

Your First Python Program

```
# This is a comment
print("Hello, World!")

# Output: Hello, World!
```

2. Python Installation & Setup

Installing Python

- Download from <https://www.python.org/downloads/>
- During installation, check 'Add Python to PATH'
- Verify installation: open terminal/cmd and type `python --version`

IDEs and Editors

- **VS Code:** Lightweight, popular, with Python extension
- **PyCharm:** Full-featured Python IDE
- **Jupyter Notebook:** Great for data science
- **IDLE:** Comes with Python by default

Running Python Code

Interactive mode: Type `python` in terminal to open the REPL.

Script mode: Save code in a `.py` file and run with `python filename.py`

Python Package Manager (pip)

```
# Install a package
pip install requests

# Install specific version
pip install numpy==1.24.0

# Upgrade a package
pip install --upgrade pandas

# List installed packages
pip list

# Uninstall a package
pip uninstall flask

# Save dependencies to file
pip freeze > requirements.txt

# Install from requirements file
pip install -r requirements.txt
```

Virtual Environments

Virtual environments allow you to manage dependencies for different projects separately.

```
# Create a virtual environment
python -m venv myenv

# Activate (Windows)
myenv\Scripts\activate

# Activate (Linux/Mac)
source myenv/bin/activate

# Deactivate
```

deactivate

3. Variables, Data Types & Operators

Variables in Python

A variable is a named location used to store data in memory. In Python, you don't need to declare the type of a variable.

```
# Variable assignment
name = "Alice"
age = 25
height = 5.7
is_student = True

# Multiple assignment
x, y, z = 1, 2, 3

# Same value to multiple variables
a = b = c = 100

# Print variables
print(name, age, height, is_student)
```

Variable Naming Rules

- Must start with a letter or underscore (`_`)
- Cannot start with a number
- Can contain letters, numbers, and underscores only
- Case-sensitive: `name` and `Name` are different
- Cannot use Python keywords (`if`, `while`, `for`, `class`, etc.)
- Use snake_case by convention: `first_name`, not `firstName`

Data Types in Python

Python has several built-in data types. The main categories are:

Category	Data Types
Numeric	int, float, complex
Sequence	list, tuple, range
Text	str
Mapping	dict
Set	set, frozenset
Boolean	bool
Binary	bytes, bytearray, memoryview
None	NoneType

Numeric Types

```
# Integer
x = 10
```



```

print(type(x)) # <class 'int'>

# Float
pi = 3.14159
print(type(pi)) # <class 'float'>

# Complex
z = 2 + 3j
print(type(z)) # <class 'complex'>

# Type conversion
a = int(3.7)      # 3
b = float(5)      # 5.0
c = str(100)      # "100"

```

Boolean Type

```

is_active = True
is_admin = False

# Booleans are subclass of int
print(True + True) # 2
print(False * 10)  # 0

# Falsy values: 0, 0.0, "", [], {}, None, False
print(bool(0))      # False
print(bool("hi"))   # True
print(bool([]))     # False

```

None Type

None represents the absence of a value.

```

result = None
if result is None:
    print("No result yet")

```

Operators in Python

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 3	3.333...
//	Floor Division	10 // 3	3
%	Modulus	10 % 3	1
**	Exponentiation	2 ** 3	8

Comparison Operators

Operator	Description	Example
==	Equal to	5 == 5 → True
!=	Not equal to	5 != 3 → True
>	Greater than	5 > 3 → True
<	Less than	5 < 3 → False
>=	Greater than or equal	5 >= 5 → True
<=	Less than or equal	3 <= 5 → True

Logical Operators

```
# and - True if both operands are True
print(True and False) # False
print(5 > 3 and 10 > 5) # True

# or - True if at least one operand is True
print(True or False) # True

# not - Inverts the boolean value
print(not True) # False
```

Assignment Operators

```
x = 10 # Simple assignment
x += 5 # x = x + 5 → 15
x -= 3 # x = x - 3 → 12
x *= 2 # x = x * 2 → 24
x /= 4 # x = x / 4 → 6.0
x //= 2 # Floor division
x %= 3 # Modulus
x **= 2 # Exponent
```

Identity & Membership Operators

```
# Identity: 'is', 'is not' - checks if same object
a = [1, 2, 3]
```

```

b = a
c = [1, 2, 3]
print(a is b)      # True (same object)
print(a is c)      # False (different objects, same values)
print(a == c)      # True (same values)

# Membership: 'in', 'not in'
fruits = ['apple', 'banana', 'cherry']
print('apple' in fruits)      # True
print('grape' not in fruits)  # True

```

Bitwise Operators

```

# & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift)
a = 12  # 1100 in binary
b = 10  # 1010 in binary
print(a & b)  # 8 (1000)
print(a | b)  # 14 (1110)
print(a ^ b)  # 6 (0110)
print(~a)     # -13
print(a << 2) # 48 (110000)
print(a >> 1) # 6 (110)

```

Input and Output

```

# Output with print()
print("Hello")
print("Name:", "Alice", "Age:", 25)
print("a", "b", "c", sep="-")      # a-b-c
print("Hello", end=" ")
print("World")                     # Hello World

# Input from user (always returns string)
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Hello {name}, you are {age} years old")

```

String Formatting

```

name = "Alice"
age = 25

# f-strings (Python 3.6+, recommended)
print(f"Name: {name}, Age: {age}")

# .format() method
print("Name: {}, Age: {}".format(name, age))
print("Name: {n}, Age: {a}".format(n=name, a=age))

# % formatting (older style)
print("Name: %s, Age: %d" % (name, age))

# Format specifiers
pi = 3.14159
print(f"{pi:.2f}")      # 3.14
print(f"{1000000:,}")   # 1,000,000
print(f"{0.85:.0%}")    # 85%

```

4. Strings in Python

Strings in Python are sequences of characters enclosed in single, double, or triple quotes. Strings are immutable.

Creating Strings

```
s1 = 'Hello'
s2 = "World"
s3 = '''Triple quoted
multi-line string'''
s4 = str(123)          # Convert to string: "123"

# String concatenation
greeting = s1 + " " + s2    # "Hello World"

# String repetition
line = "-" * 20             # "-----"
```

String Indexing and Slicing

```
s = "Python"

# Indexing (0-based)
print(s[0])    # 'P'
print(s[5])    # 'n'
print(s[-1])   # 'n' (negative indexing)
print(s[-2])   # 'o'

# Slicing [start:stop:step]
print(s[0:3])  # 'Pyt'
print(s[:3])   # 'Pyt'
print(s[2:])   # 'thon'
print(s[:2])   # 'Pt'
print(s[::-1]) # 'nohtyP' (reverse)
```

Common String Methods

Method	Description	Example
upper()	Convert to uppercase	"hi".upper() → "HI"
lower()	Convert to lowercase	"HI".lower() → "hi"
title()	Title case	"hi there".title() → "Hi There"
capitalize()	First letter uppercase	"hello".capitalize() → "Hello"
strip()	Remove whitespace	" hi ".strip() → "hi"
lstrip()/rstrip()	Left/right strip	" hi ".lstrip() → "hi "
split()	Split into list	"a,b,c".split(",") → ["a","b","c"]
join()	Join list into string	", ".join(["a","b"]) → "a,b"
replace()	Replace substring	"hi".replace("h","H") → "Hi"
find()	Find substring index	"hello".find("ll") → 2
index()	Like find, raises error	"hello".index("l") → 2

count()	Count occurrences	"banana".count("a") → 3
startswith()	Check prefix	"hello".startswith("he") → True
endswith()	Check suffix	"hello".endswith("lo") → True
isdigit()	All digits?	"123".isdigit() → True
isalpha()	All letters?	"abc".isalpha() → True
isalnum()	All alphanumeric?	"abc123".isalnum() → True
len()	Length of string	len("hello") → 5

String Formatting Examples

```

name, age = "Bob", 30

# f-string (modern, recommended)
msg = f"My name is {name} and I am {age} years old."

# Expressions inside f-strings
print(f"Next year: {age + 1}")
print(f"Upper: {name.upper()}")

# Width and alignment
print(f"{'Left':<10}|")      # Left:      |
print(f"{'Right':>10}|")    #      Right|
print(f"{'Center':^10}|")   #    Center |

# Number formatting
n = 12345.6789
print(f"{n:.2f}")           # 12345.68
print(f"{n:,.2f}")          # 12,345.68
print(f"{n:e}")              # 1.234568e+04
print(f"{42:05d}")           # 00042 (padded)
print(f"{42:b}")             # 101010 (binary)
print(f"{42:o}")             # 52 (octal)
print(f"{42:x}")             # 2a (hex)

```

Escape Characters

Sequence	Meaning
\n	New line
\t	Tab
\\	Backslash
\'	Single quote
\"	Double quote
\r	Carriage return
\b	Backspace

```

# Raw strings (no escape processing)
path = r"C:\Users\Alice\Documents"
print(path)  # C:\Users\Alice\Documents

```

```
# Multiline strings
text = """Line 1
Line 2
Line 3"""
```

String Immutability

Strings cannot be changed after creation. Any 'modification' creates a new string.

```
s = "hello"
# s[0] = 'H' # Error! Strings are immutable

# Create new string instead
s = "H" + s[1:] # "Hello"
s = s.replace('h', 'H') # Returns new string
```

5. Control Flow Statements

Conditional Statements

Used to execute code based on conditions. Python uses indentation (typically 4 spaces) to define blocks.

if Statement

```
age = 18

if age >= 18:
    print("You are an adult")
```

if-else Statement

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

if-elif-else Statement

```
score = 75

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'

print(f"Grade: {grade}") # Grade: C
```

Nested if Statements

```
num = 10

if num > 0:
    if num % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
else:
    print("Non-positive number")
```

Ternary (Conditional Expression)

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status) # Adult

# More complex
n = 5
result = "positive" if n > 0 else "negative" if n < 0 else "zero"
```

Loops

Loops execute a block of code repeatedly. Python has **for** and **while** loops.

for Loop

```
# Iterate over a sequence
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)

# Using range()
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):       # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)

# Iterate over string
for char in "Python":
    print(char)

# Iterate with index
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(index, fruit)

# Iterate over dict
person = {"name": "Alice", "age": 25}
for key, value in person.items():
    print(key, ":", value)

# Iterate two lists in parallel
names = ["Alice", "Bob"]
ages = [25, 30]
for name, age in zip(names, ages):
    print(name, age)
```

while Loop

```
# Run while condition is True
count = 0
while count < 5:
    print(count)
    count += 1

# Infinite loop with break
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == 'quit':
        break
    print(f"You entered: {user_input}")
```

Loop Control Statements

```
# break - exit the loop completely
for i in range(10):
    if i == 5:
        break
    print(i)
# Output: 0, 1, 2, 3, 4

# continue - skip current iteration
for i in range(10):
    if i % 2 == 0:
        continue
```



```

    print(i)
# Output: 1, 3, 5, 7, 9

# pass - do nothing (placeholder)
for i in range(5):
    pass # Used when syntax requires a statement

# else clause with loops (executes if loop completes without break)
for i in range(5):
    print(i)
else:
    print("Loop completed normally")

```

Nested Loops

```

# Multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i * j:4}", end=" ")
    print()

# Pattern printing
for i in range(5):
    for j in range(i + 1):
        print("*", end=" ")
    print()

# Output:
# *
# * *
# * * *
# * * * *
# * * * * *

```

6. Functions

A function is a reusable block of code that performs a specific task. Functions help in code reusability and modularity.

Defining and Calling Functions

```
# Define a function
def greet():
    print("Hello, World!")

# Call the function
greet()

# Function with parameters
def greet_person(name):
    print(f"Hello, {name}!")

greet_person("Alice")

# Function with return value
def add(a, b):
    return a + b

result = add(5, 3)
print(result) # 8
```

Types of Arguments

1. Positional Arguments

```
def describe_pet(name, animal):
    print(f"{name} is a {animal}")

describe_pet("Rex", "dog")    # Rex is a dog
```

2. Keyword Arguments

```
describe_pet(animal="cat", name="Whiskers")
# Order doesn't matter when using keywords
```

3. Default Arguments

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")           # Hello, Alice!
greet("Bob", "Hi")       # Hi, Bob!
greet("Carol", greeting="Hey") # Hey, Carol!
```

4. Variable-Length Arguments

```
# *args - variable positional arguments (tuple)
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_all(1, 2, 3))    # 6
print(sum_all(1, 2, 3, 4, 5)) # 15

# **kwargs - variable keyword arguments (dict)
def show_info(**details):
```

```

        for key, value in details.items():
            print(f"{key}: {value}")

show_info(name="Alice", age=25, city="NY")

# Both together
def example(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)

example(1, 2, 3, name="Alice", age=25)
# args: (1, 2, 3)
# kwargs: {'name': 'Alice', 'age': 25}

```

Return Values

```

# Return single value
def square(n):
    return n * n

# Return multiple values (as tuple)
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([3, 1, 4, 1, 5, 9, 2])
print(low, high) # 1 9

# Function with no return returns None
def greet(name):
    print(f"Hi {name}")

result = greet("Alice")
print(result) # None

```

Scope of Variables

Python has the LEGB rule for variable scope: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in.

```

x = 10 # Global variable

def outer():
    y = 20 # Enclosing variable

    def inner():
        z = 30 # Local variable
        print(x, y, z)

    inner()

outer() # 10 20 30

# Modify global variable
count = 0

def increment():
    global count
    count += 1

increment()
print(count) # 1

# Nonlocal (for nested functions)
def outer():
    x = 10
    def inner():

```

```

        nonlocal x
        x = 20
    inner()
    print(x) # 20

```

Lambda Functions (Anonymous Functions)

```

# Syntax: lambda arguments: expression
square = lambda x: x * x
print(square(5)) # 25

add = lambda a, b: a + b
print(add(3, 4)) # 7

# Common use with sorted, filter, map
numbers = [(1, 2), (3, 1), (5, 0)]
sorted_nums = sorted(numbers, key=lambda x: x[1])
print(sorted_nums) # [(5, 0), (3, 1), (1, 2)]

```

Recursive Functions

A function that calls itself. Must have a base case to avoid infinite recursion.

```

# Factorial using recursion
def factorial(n):
    if n <= 1:          # Base case
        return 1
    return n * factorial(n - 1) # Recursive case

print(factorial(5)) # 120

# Fibonacci sequence
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(7)) # 13

```

Docstrings

```

def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of rectangle
        width (float): The width of rectangle

    Returns:
        float: Area of the rectangle
    """
    return length * width

# Access docstring
print(calculate_area.__doc__)
help(calculate_area)

```

Function as First-Class Objects

In Python, functions are first-class citizens — they can be assigned to variables, passed as arguments, and returned from other functions.

```
def greet(name):
    return f"Hello, {name}"

# Assign to variable
say_hi = greet
print(say_hi("Alice")) # Hello, Alice

# Pass as argument
def execute(func, arg):
    return func(arg)

print(execute(greet, "Bob")) # Hello, Bob

# Return from function
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5)) # 10
print(triple(5)) # 15
```

7. Data Structures: Lists, Tuples, Sets, Dictionaries

Lists

Lists are ordered, mutable (changeable), allow duplicates, and can contain different data types.

```
# Creating lists
empty = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True, [1, 2]]
fruits = ["apple", "banana", "cherry"]

# Accessing elements
print(fruits[0])      # apple
print(fruits[-1])     # cherry
print(fruits[1:3])    # ['banana', 'cherry']

# Modifying lists
fruits[0] = "orange"  # Mutable
fruits.append("mango") # Add to end
fruits.insert(1, "grape") # Insert at index
fruits.extend(["kiwi", "pear"]) # Add multiple
fruits.remove("banana") # Remove by value
fruits.pop()          # Remove last (or by index)
del fruits[0]          # Delete by index
fruits.clear()         # Remove all

# Useful methods
nums = [3, 1, 4, 1, 5, 9, 2, 6]
print(len(nums))      # 8
print(min(nums))      # 1
print(max(nums))      # 9
print(sum(nums))      # 31
print(nums.count(1))  # 2
print(nums.index(5))  # 4
nums.sort()           # Sort in place
nums.reverse()        # Reverse in place
sorted_nums = sorted(nums) # Return new sorted list

# List concatenation and repetition
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)          # [1, 2, 3, 4, 5, 6]
print(a * 3)          # [1, 2, 3, 1, 2, 3, 1, 2, 3]

# Membership testing
print(3 in a)         # True

# Copy a list
copy1 = a.copy()
copy2 = list(a)
copy3 = a[:]
```

Tuples

Tuples are ordered, **immutable**, allow duplicates. Use when data shouldn't change.

```
# Creating tuples
empty = ()
single = (5,)          # Note the comma!
coords = (10, 20)
mixed = (1, "hello", 3.14)
```

```

nested = ((1, 2), (3, 4))

# Without parentheses (tuple packing)
point = 1, 2, 3

# Accessing elements (same as lists)
print(coords[0])      # 10
print(coords[-1])     # 20

# Tuples are immutable
# coords[0] = 100 # Error!

# Tuple unpacking
x, y = coords
print(x, y)           # 10 20

# Swap variables using tuples
a, b = 5, 10
a, b = b, a
print(a, b)           # 10 5

# Tuple methods
t = (1, 2, 3, 2, 1)
print(t.count(2))     # 2
print(t.index(3))     # 2
print(len(t))         # 5

# When to use tuples?
# - Returning multiple values from a function
# - Dictionary keys (lists can't be keys)
# - Data that should not change (coordinates, RGB values)

```

Sets

Sets are unordered, mutable, contain **unique** elements (no duplicates). Useful for removing duplicates and set operations.

```

# Creating sets
empty = set()          # Note: {} creates empty dict
fruits = {"apple", "banana", "cherry"}
nums = set([1, 2, 3, 2, 1]) # {1, 2, 3}

# Adding and removing
fruits.add("orange")
fruits.update(["kiwi", "mango"])
fruits.remove("apple")    # Error if not present
fruits.discard("grape")   # No error if not present
fruits.pop()              # Remove random element
fruits.clear()            # Remove all

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)              # Union: {1, 2, 3, 4, 5, 6}
print(a & b)               # Intersection: {3, 4}
print(a - b)              # Difference: {1, 2}
print(a ^ b)              # Symmetric difference: {1, 2, 5, 6}

# Equivalent methods
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
print(a.symmetric_difference(b))

```

```

# Subset and superset
print({1, 2}.issubset(a))      # True
print(a.issuperset({1, 2}))   # True

# Remove duplicates from list
nums = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(nums))      # [1, 2, 3, 4]

# Frozen set (immutable set)
fs = frozenset([1, 2, 3])

```

Dictionaries

Dictionaries store key-value pairs. Keys must be unique and immutable (strings, numbers, tuples). Values can be any type.

```

# Creating dictionaries
empty = {}
person = {"name": "Alice", "age": 25, "city": "NYC"}
person2 = dict(name="Bob", age=30)

# Accessing values
print(person["name"])          # Alice
print(person.get("age"))       # 25
print(person.get("phone", "N/A")) # N/A (default)

# Modifying
person["age"] = 26             # Update
person["email"] = "a@a.com"    # Add new key
person.update({"phone": "123", "age": 27})

# Removing
del person["city"]
removed = person.pop("phone")  # Returns value
person.popitem()               # Remove last inserted
person.clear()                 # Remove all

# Useful methods
person = {"name": "Alice", "age": 25, "city": "NYC"}
print(person.keys())           # dict_keys(['name', 'age', 'city'])
print(person.values())         # dict_values(['Alice', 25, 'NYC'])
print(person.items())          # dict_items([('name', 'Alice'), ...])
print(len(person))             # 3
print("name" in person)        # True

# Iterating
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")

# Nested dictionaries
employees = {
    "emp1": {"name": "Alice", "salary": 50000},
    "emp2": {"name": "Bob", "salary": 60000}
}
print(employees["emp1"]["name"]) # Alice

# Dictionary comprehension
squares = {x: x*x for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Merge dictionaries (Python 3.9+)
d1 = {"a": 1, "b": 2}

```



```
d2 = {"c": 3, "d": 4}
merged = d1 | d2
```

Comparison Table

Feature	List	Tuple	Set	Dict
Syntax	[1,2,3]	(1,2,3)	{1,2,3}	{"a":1}
Ordered	Yes	Yes	No*	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys: No
Indexable	Yes	Yes	No	By key
Use case	Sequence	Fixed data	Unique items	Key-value

* *Insertion-ordered in CPython 3.7+, but you shouldn't rely on it.*

8. File Handling

Python provides built-in functions to read from and write to files.

Opening Files

```
# Syntax: open(filename, mode)
# Modes:
# 'r' - read (default)
# 'w' - write (overwrites if exists)
# 'a' - append
# 'x' - exclusive creation (fails if exists)
# 'b' - binary mode (rb, wb)
# 't' - text mode (default)
# '+' - read and write (r+, w+, a+)

file = open("example.txt", "r")
content = file.read()
file.close() # Always close!
```

Using with Statement (Recommended)

The with statement automatically closes the file.

```
# Reading a file
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
# File is automatically closed here

# Read line by line (memory efficient)
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())

# Read all lines into a list
with open("example.txt", "r") as file:
    lines = file.readlines()

# Read one line
with open("example.txt", "r") as file:
    first_line = file.readline()
```

Writing to Files

```
# Write (overwrites existing content)
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Second line\n")

# Write multiple lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)

# Append to file
with open("output.txt", "a") as file:
    file.write("Appended line\n")
```

Working with CSV Files

```
import csv
```

```

# Reading CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Reading as dictionary
with open("data.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], row["age"])

# Writing CSV
data = [
    ["Name", "Age", "City"],
    ["Alice", 25, "NY"],
    ["Bob", 30, "LA"]
]
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)

```

Working with JSON Files

```

import json

# Python dict to JSON string
data = {"name": "Alice", "age": 25, "city": "NY"}
json_str = json.dumps(data, indent=2)
print(json_str)

# Write to JSON file
with open("data.json", "w") as file:
    json.dump(data, file, indent=2)

# Read JSON file
with open("data.json", "r") as file:
    data = json.load(file)
    print(data)

# Parse JSON string
json_str = '{"name": "Alice", "age": 25}'
data = json.loads(json_str)
print(data["name"])

```

File and Directory Operations

```

import os

# Check if file exists
if os.path.exists("file.txt"):
    print("File exists")

# Get file size
size = os.path.getsize("file.txt")

# Rename file
os.rename("old.txt", "new.txt")

# Delete file
os.remove("file.txt")

# Create directory
os.mkdir("new_folder")
os.makedirs("path/to/folder") # Create intermediate

```

```
# List directory
files = os.listdir(".")

# Delete directory
os.rmdir("empty_folder")

# Current directory
print(os.getcwd())
os.chdir("/path/to/dir")

# Using pathlib (modern, recommended)
from pathlib import Path

p = Path("example.txt")
print(p.exists())
print(p.is_file())
print(p.suffix)          # .txt
print(p.stem)           # example
p.read_text()
p.write_text("Hello")
```

9. Exception Handling

Exceptions are runtime errors. Exception handling prevents your program from crashing when errors occur.

Common Built-in Exceptions

Exception	Cause
SyntaxError	Invalid Python syntax
NameError	Variable not defined
TypeError	Wrong type operation
ValueError	Wrong value for operation
IndexError	List index out of range
KeyError	Dictionary key not found
AttributeError	Attribute doesn't exist
ZeroDivisionError	Division by zero
FileNotFoundError	File doesn't exist
ImportError	Module import failed
IndentationError	Improper indentation
RuntimeError	Generic runtime error
StopIteration	Iterator exhausted

try-except Block

```
# Basic try-except
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")

# Multiple exceptions
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")

# Multiple exceptions in one except
try:
    pass # some code
except (ValueError, TypeError) as e:
    print(f"Error: {e}")

# Catch any exception (use carefully)
try:
    pass
```

```
except Exception as e:
    print(f"Error: {e}")
    print(f"Type: {type(e).__name__}")
```

try-except-else-finally

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
else:
    # Runs only if no exception
    print("File read successfully")
    print(content)
finally:
    # Always runs (cleanup code)
    if 'file' in locals() and not file.closed:
        file.close()
    print("Cleanup done")
```

Raising Exceptions

```
# Raise built-in exception
def divide(a, b):
    if b == 0:
        raise ValueError("Divisor cannot be zero")
    return a / b

try:
    result = divide(10, 0)
except ValueError as e:
    print(e)

# Re-raise exception
try:
    raise ValueError("Original error")
except ValueError:
    print("Logged")
    raise # Re-raise the same exception
```

Custom Exceptions

```
class InsufficientBalanceError(Exception):
    """Raised when bank account has insufficient balance"""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        message = f"Cannot withdraw {amount}; balance is {balance}"
        super().__init__(message)

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    new_balance = withdraw(100, 500)
except InsufficientBalanceError as e:
    print(e)
    print(f"Short by: {e.amount - e.balance}")
```

assert Statement

```
# Assert checks if a condition is true
def calculate_age(birth_year, current_year):
    assert birth_year <= current_year, "Birth year must be <= current year"
    return current_year - birth_year

# Throws AssertionError if condition fails
# Note: Asserts are removed in optimized mode (python -O)
```

10. Object-Oriented Programming (OOP)

OOP is a programming paradigm based on the concept of objects, which contain data (attributes) and code (methods). The four pillars of OOP are **Encapsulation, Inheritance, Polymorphism, and Abstraction**.

Classes and Objects

```
# Define a class
class Dog:
    # Class attribute (shared by all instances)
    species = "Canis familiaris"

    # Constructor (called when creating object)
    def __init__(self, name, age):
        # Instance attributes
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        return f"{self.name} says Woof!"

    def get_info(self):
        return f"{self.name} is {self.age} years old"

# Create objects (instances)
dog1 = Dog("Rex", 3)
dog2 = Dog("Buddy", 5)

# Access attributes
print(dog1.name)          # Rex
print(dog1.species)       # Canis familiaris

# Call methods
print(dog1.bark())        # Rex says Woof!
print(dog2.get_info())    # Buddy is 5 years old

# Modify attributes
dog1.age = 4
dog1.color = "Brown"      # Add new attribute
```

The Four Pillars of OOP

1. Encapsulation

Bundling data and methods that work on that data within a class, restricting direct access. Achieved using access modifiers.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          # Public
        self._balance = balance     # Protected (convention)
        self.__pin = "1234"         # Private (name-mangled)

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
            return amount
```



```

    def get_balance(self):
        return self._balance

account = BankAccount("Alice", 1000)
account.deposit(500)
print(account.get_balance()) # 1500

# Private attribute name-mangling
# print(account.__pin) # AttributeError
print(account._BankAccount__pin) # 1234 (not truly private)

```

2. Inheritance

A class can inherit attributes and methods from another class. Promotes code reusability.

```

# Parent class (Base class)
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

    def info(self):
        return f"I am {self.name}"

# Child class (Derived class)
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Call parent constructor
        self.breed = breed

    # Override parent method
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

dog = Dog("Rex", "Labrador")
print(dog.info()) # I am Rex
print(dog.speak()) # Woof!
print(dog.breed) # Labrador

# Check inheritance
print(isinstance(dog, Dog)) # True
print(isinstance(dog, Animal)) # True
print(issubclass(Dog, Animal)) # True

```

Types of Inheritance

```

# 1. Single Inheritance
class A: pass
class B(A): pass

# 2. Multiple Inheritance
class Father:
    def skills(self):
        return "Coding"

class Mother:
    def hobbies(self):
        return "Painting"

class Child(Father, Mother):
    pass

```

```

c = Child()
print(c.skills(), c.hobbies())

# 3. Multilevel Inheritance
class Grandfather:
    pass
class Father(Grandfather):
    pass
class Son(Father):
    pass

# 4. Hierarchical Inheritance
class Vehicle:
    pass
class Car(Vehicle):
    pass
class Bike(Vehicle):
    pass

# Method Resolution Order (MRO)
print(Child.__mro__)
# Child -> Father -> Mother -> object

```

3. Polymorphism

'Many forms' - the same method name can behave differently for different classes.

```

class Bird:
    def fly(self):
        return "Bird flies"

class Eagle(Bird):
    def fly(self):
        return "Eagle soars high"

class Penguin(Bird):
    def fly(self):
        return "Penguin cannot fly"

# Same method call, different behavior
for bird in [Bird(), Eagle(), Penguin()]:
    print(bird.fly())

# Function polymorphism
def add(x, y):
    return x + y

print(add(2, 3))           # 5 (numbers)
print(add("Hi ", "there")) # Hi there (strings)
print(add([1,2], [3,4]))   # [1,2,3,4] (lists)

# Operator overloading
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2) # Vector(4, 6)

```

4. Abstraction

Hiding complex implementation details and showing only the essential features. Implemented using abstract classes.

```
from abc import ABC, abstractmethod

# Abstract class
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14159 * self.radius

# shape = Shape() # TypeError! Cannot instantiate abstract class
r = Rectangle(5, 3)
print(r.area(), r.perimeter())
```

Special (Dunder/Magic) Methods

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        # User-friendly string (used by print)
        return f"Book: {self.title}"

    def __repr__(self):
        # Developer-friendly representation
        return f"Book('{self.title}', {self.pages})"

    def __len__(self):
        # Used by len()
        return self.pages

    def __eq__(self, other):
        # Used by ==
        return self.title == other.title

    def __lt__(self, other):
```

```

        # Used by <
        return self.pages < other.pages

book1 = Book("Python", 300)
book2 = Book("Java", 400)

print(book1)           # Book: Python
print(repr(book1))     # Book('Python', 300)
print(len(book1))      # 300
print(book1 < book2)   # True

```

Class Methods and Static Methods

```

class MathOperations:
    pi = 3.14159

    def __init__(self, value):
        self.value = value

    # Instance method (takes self)
    def square(self):
        return self.value ** 2

    # Class method (takes cls)
    @classmethod
    def get_pi(cls):
        return cls.pi

    # Static method (no self or cls)
    @staticmethod
    def add(a, b):
        return a + b

# Using them
m = MathOperations(5)
print(m.square())           # 25 (instance method)
print(MathOperations.get_pi()) # 3.14159 (class method)
print(MathOperations.add(2,3)) # 5 (static method)

```

Property Decorators (Getters/Setters)

```

class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        """Getter"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Setter"""
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):
        """Computed property"""
        return self._celsius * 9/5 + 32

t = Temperature(25)
print(t.celsius)           # 25 (using getter)
t.celsius = 30              # using setter

```

```
print(t.fahrenheit)    # 86.0
```

11. Modules and Packages

A **module** is a Python file (.py) containing definitions and statements. A **package** is a collection of modules organized in directories.

Importing Modules

```
# Import entire module
import math
print(math.pi)
print(math.sqrt(16))

# Import with alias
import numpy as np
arr = np.array([1, 2, 3])

# Import specific functions
from math import pi, sqrt
print(pi)
print(sqrt(16))

# Import all (not recommended)
from math import *

# Import with alias
from datetime import datetime as dt
```

Creating Your Own Module

Save as mymodule.py:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159

class Calculator:
    def add(self, a, b):
        return a + b
```

Use it in another file:

```
# main.py
import mymodule

print(mymodule.greet("Alice"))
print(mymodule.PI)
calc = mymodule.Calculator()
```

Packages

A package is a directory with an `__init__.py` file and modules.

```
mypackage/
  __init__.py
  module1.py
  module2.py
  subpackage/
    __init__.py
    module3.py

# Using the package
```

```

from mypackage import module1
from mypackage.subpackage import module3
import mypackage.module1 as m1

```

Useful Built-in Modules

Module	Purpose
math	Mathematical functions (sqrt, pi, sin)
random	Random number generation
datetime	Date and time operations
os	Operating system interface
sys	System-specific parameters
json	JSON parsing and serialization
re	Regular expressions
collections	Specialized container types
itertools	Iterator building blocks
functools	Functional programming tools
time	Time-related functions
csv	CSV file reading and writing
urllib	URL handling
sqlite3	SQLite database
threading	Thread-based parallelism
multiprocessing	Process-based parallelism

`__name__ == "__main__"`

Used to check if a file is run directly or imported.

```

# script.py
def main():
    print("Running directly")

if __name__ == "__main__":
    main()
# main() runs only when script.py is run directly,
# not when imported as a module

```

12. Iterators and Generators

Iterators

An iterator is an object that can be iterated upon, returning one element at a time. Implements `__iter__()` and `__next__()`.

```
# Using an iterator
nums = [1, 2, 3]
it = iter(nums)
print(next(it))    # 1
print(next(it))    # 2
print(next(it))    # 3
# print(next(it))  # StopIteration

# Custom iterator
class CountUpTo:
    def __init__(self, max_value):
        self.max_value = max_value
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.max_value:
            raise StopIteration
        self.current += 1
        return self.current

for num in CountUpTo(5):
    print(num)      # 1, 2, 3, 4, 5
```

Generators

Generators are a simpler way to create iterators using the `yield` keyword. They are memory-efficient — they generate values on the fly.

```
# Generator function
def count_up_to(max_value):
    n = 1
    while n <= max_value:
        yield n
        n += 1

for num in count_up_to(5):
    print(num)      # 1, 2, 3, 4, 5

# Generator preserves state between calls
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib), end=" ")
# 0 1 1 2 3 5 8 13 21 34

# Generator expression (like list comprehension)
squares = (x*x for x in range(10))
print(next(squares))  # 0
```



```

print(sum(squares))    # 285

# Memory efficiency comparison
# List: stores all elements
big_list = [x for x in range(10_000_000)] # Uses lots of memory

# Generator: generates on demand
big_gen = (x for x in range(10_000_000)) # Uses little memory

```

Common Uses of Generators

```

# Reading large files
def read_large_file(filename):
    with open(filename) as f:
        for line in f:
            yield line.strip()

for line in read_large_file("huge_file.txt"):
    process(line)

# Infinite sequences
def counter(start=0):
    n = start
    while True:
        yield n
        n += 1

# Pipeline of generators
def squared(nums):
    for n in nums:
        yield n * n

def even(nums):
    for n in nums:
        if n % 2 == 0:
            yield n

result = list(even(squared(range(10))))
# [0, 4, 16, 36, 64]

```

13. Decorators

A decorator is a function that takes another function and extends or modifies its behavior without explicitly modifying it. They use the @ symbol.

Basic Decorator

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Output:
# Before function call
# Hello!
# After function call

# Equivalent without @ syntax:
say_hello = my_decorator(say_hello)
```

Decorator with Arguments

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@my_decorator
def add(a, b):
    return a + b

print(add(2, 3))
# Output:
# Calling add
# Finished add
# 5
```

Practical Examples

Timing Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper

@timer
```

```
def slow_function():
    time.sleep(1)
    return "Done"

slow_function() # slow_function took 1.0012s
```

Logging Decorator

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"[LOG] {func.__name__}({args}, {kwargs})")
        return func(*args, **kwargs)
    return wrapper

@log_calls
def divide(a, b):
    return a / b

divide(10, 2)
# [LOG] divide((10, 2), {})
```

Authentication Decorator

```
def require_auth(func):
    def wrapper(user, *args, **kwargs):
        if not user.get("authenticated"):
            return "Access denied"
        return func(user, *args, **kwargs)
    return wrapper

@require_auth
def view_dashboard(user):
    return f"Welcome, {user['name']}"
```

```
user1 = {"name": "Alice", "authenticated": True}
user2 = {"name": "Bob", "authenticated": False}
print(view_dashboard(user1)) # Welcome, Alice
print(view_dashboard(user2)) # Access denied
```

Decorators with Parameters

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello, {name}")

greet("Alice")
# Hello, Alice
# Hello, Alice
# Hello, Alice
```

Built-in Decorators

```
# @staticmethod, @classmethod, @property (already covered in OOP)

# @functools.wraps - preserve original function metadata
from functools import wraps
```

```

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def hello():
    """Says hello"""
    pass

print(hello.__name__) # hello (not 'wrapper')
print(hello.__doc__)  # Says hello

# @lru_cache - memoization
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(50)) # Fast, due to caching

```

14. Lambda, Map, Filter, Reduce

Lambda Functions

Anonymous, one-line functions defined with `lambda` keyword.

```
# Syntax: lambda arguments: expression
square = lambda x: x ** 2
print(square(5))    # 25

add = lambda x, y: x + y
print(add(3, 4))    # 7

# Conditional expression
max_val = lambda a, b: a if a > b else b
print(max_val(10, 20)) # 20

# Cannot have multiple statements or return
# Use regular def for complex logic
```

map() Function

Applies a function to all items in an iterable. Returns a map object (iterator).

```
# Syntax: map(function, iterable)
numbers = [1, 2, 3, 4, 5]

# With lambda
squared = list(map(lambda x: x**2, numbers))
print(squared) # [1, 4, 9, 16, 25]

# With regular function
def double(x):
    return x * 2

doubled = list(map(double, numbers))
print(doubled) # [2, 4, 6, 8, 10]

# Multiple iterables
a = [1, 2, 3]
b = [10, 20, 30]
result = list(map(lambda x, y: x + y, a, b))
print(result) # [11, 22, 33]

# Convert strings to integers
str_nums = ["1", "2", "3"]
int_nums = list(map(int, str_nums))
print(int_nums) # [1, 2, 3]
```

filter() Function

Filters elements that satisfy a condition. Returns a filter object.

```
# Syntax: filter(function, iterable)
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Get even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6, 8, 10]

# Filter strings
words = ["apple", "banana", "cat", "dog", "elephant"]
long_words = list(filter(lambda w: len(w) > 3, words))
```

```

print(long_words) # ['apple', 'banana', 'elephant']

# Remove None values
data = [1, None, 2, None, 3]
clean = list(filter(None, data))
print(clean) # [1, 2, 3]

```

reduce() Function

Reduces an iterable to a single value by applying a function cumulatively.

```

from functools import reduce

# Syntax: reduce(function, iterable[, initial])
numbers = [1, 2, 3, 4, 5]

# Sum all numbers
total = reduce(lambda x, y: x + y, numbers)
print(total) # 15

# Product of all numbers
product = reduce(lambda x, y: x * y, numbers)
print(product) # 120

# Find maximum
max_num = reduce(lambda x, y: x if x > y else y, numbers)
print(max_num) # 5

# With initial value
total = reduce(lambda x, y: x + y, numbers, 100)
print(total) # 115 (100 + 15)

```

Practical Comparison

```

nums = [1, 2, 3, 4, 5]

# Get squares of even numbers
result = list(map(
    lambda x: x**2,
    filter(lambda x: x % 2 == 0, nums)
))
print(result) # [4, 16]

# Same with list comprehension (more Pythonic)
result = [x**2 for x in nums if x % 2 == 0]
print(result) # [4, 16]

```

15. Comprehensions

Comprehensions provide a concise way to create lists, sets, and dictionaries.

List Comprehension

```
# Syntax: [expression for item in iterable if condition]

# Simple
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With condition
even_squares = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

# With if-else (expression)
labels = ["even" if x % 2 == 0 else "odd" for x in range(5)]
# ['even', 'odd', 'even', 'odd', 'even']

# Nested loops
pairs = [(x, y) for x in [1, 2, 3] for y in ['a', 'b']]
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b'), (3, 'a'), (3, 'b')]

# Matrix flattening
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Matrix transpose
transposed = [[row[i] for row in matrix] for i in range(3)]
# [[1,4,7], [2,5,8], [3,6,9]]

# String manipulation
words = ["hello", "world"]
upper = [w.upper() for w in words]
# ['HELLO', 'WORLD']
```

Set Comprehension

```
# Same syntax but with {} instead of []
unique_squares = {x**2 for x in [1, 2, 2, 3, 3, 3]}
# {1, 4, 9}

# Remove duplicates from list
nums = [1, 2, 2, 3, 4, 4, 5]
unique = {x for x in nums}
# {1, 2, 3, 4, 5}
```

Dictionary Comprehension

```
# Syntax: {key_expr: value_expr for item in iterable if condition}

# Simple
squares_dict = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# From two lists
keys = ['a', 'b', 'c']
values = [1, 2, 3]
d = {k: v for k, v in zip(keys, values)}
# {'a': 1, 'b': 2, 'c': 3}
```

```
# Filter dictionary
prices = {"apple": 0.5, "banana": 0.3, "cherry": 2.0, "date": 5.0}
cheap = {k: v for k, v in prices.items() if v < 1.0}
# {'apple': 0.5, 'banana': 0.3}

# Swap keys and values
swapped = {v: k for k, v in {"a": 1, "b": 2}.items()}
# {1: 'a', 2: 'b'}
```

Generator Expression

```
# Like list comprehension but with () - memory efficient
gen = (x**2 for x in range(10))
print(next(gen))    # 0
print(sum(gen))     # 285

# When passing to functions
sum_squares = sum(x**2 for x in range(10)) # No need for inner ()
```

Note: List comprehensions are usually faster and more Pythonic than map/filter with lambda. However, for complex logic, regular loops are more readable.

16. Regular Expressions

Regular expressions (regex) are patterns used to match character combinations in strings. Python's `re` module provides regex support.

Basic Patterns

Pattern	Meaning
.	Any character except newline
^	Start of string
\$	End of string
*	0 or more repetitions
+	1 or more repetitions
?	0 or 1 repetition
{n}	Exactly n repetitions
{n,m}	Between n and m repetitions
[abc]	Any of a, b, or c
[^abc]	NOT a, b, or c
[a-z]	Range a to z
\d	Digit [0-9]
\D	Non-digit
\w	Word character [a-zA-Z0-9_]
\W	Non-word character
\s	Whitespace
\S	Non-whitespace
	OR (alternation)
()	Grouping
\b	Word boundary

Common Functions

```
import re

# re.match() - match at the BEGINNING of string
result = re.match(r'\d+', '123abc')
print(result.group())    # 123

# re.search() - find FIRST match anywhere
result = re.search(r'\d+', 'abc123def')
print(result.group())    # 123
```

```

# re.findall() - find ALL matches as list
matches = re.findall(r'\d+', '1 cat, 2 dogs, 3 birds')
print(matches)    # ['1', '2', '3']

# re.finditer() - find all matches as iterator
for m in re.finditer(r'\d+', 'a1 b2 c3'):
    print(m.group(), m.start())

# re.sub() - substitute
result = re.sub(r'\d+', 'NUM', 'abc 123 def 456')
print(result)    # abc NUM def NUM

# re.split() - split by pattern
parts = re.split(r'[;,]', 'a,b;c,d;e')
print(parts)    # ['a', 'b', 'c', 'd', 'e']

# Compile pattern for reuse
pattern = re.compile(r'\d+')
print(pattern.findall('1 2 3'))    # ['1', '2', '3']

```

Practical Examples

```

import re

# Email validation
email_pattern = r'^[\w.-]+@[\w.-]+\.\w+$'
def is_valid_email(email):
    return bool(re.match(email_pattern, email))

print(is_valid_email("alice@example.com"))    # True
print(is_valid_email("invalid.email"))        # False

# Phone number extraction
text = "Call me at 555-1234 or 555-5678"
phones = re.findall(r'\d{3}-\d{4}', text)
print(phones)    # ['555-1234', '555-5678']

# URL extraction
text = "Visit https://example.com or http://test.org"
urls = re.findall(r'https?://[\w./]+', text)
print(urls)

# Capture groups
date = "2023-10-15"
match = re.match(r'(\d{4})-(\d{2})-(\d{2})', date)
if match:
    year, month, day = match.groups()
    print(f"Year: {year}, Month: {month}, Day: {day}")

# Named groups
match = re.match(r'(?P<year>\d{4})-(?P<month>\d{2})', '2023-10')
print(match.group('year'))    # 2023
print(match.group('month'))    # 10

# Flags
text = "HELLO World"
re.findall(r'hello', text, re.IGNORECASE)    # ['HELLO']

```

17. Multithreading & Multiprocessing

Used for concurrent execution. Threads run in the same memory space, processes have separate memory spaces.

Multithreading

Best for I/O-bound tasks (network requests, file I/O). Due to Python's **GIL** (Global Interpreter Lock), threads don't provide true parallelism for CPU-bound tasks.

```
import threading
import time

def task(name, delay):
    print(f"Thread {name} starting")
    time.sleep(delay)
    print(f"Thread {name} finished")

# Create threads
t1 = threading.Thread(target=task, args=("A", 2))
t2 = threading.Thread(target=task, args=("B", 1))

# Start threads
t1.start()
t2.start()

# Wait for threads to complete
t1.join()
t2.join()

print("All threads done")

# Thread with lock (avoid race conditions)
counter = 0
lock = threading.Lock()

def increment():
    global counter
    with lock:
        for _ in range(100000):
            counter += 1

threads = [threading.Thread(target=increment) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print(counter)    # 500000
```

Multiprocessing

Best for CPU-bound tasks. Bypasses GIL by using multiple processes.

```
from multiprocessing import Process, Pool
import os

def task(name):
    print(f"Process {name}, PID: {os.getpid()}")

if __name__ == "__main__":
    processes = []
    for i in range(3):
        p = Process(target=task, args=(i,))
        processes.append(p)
        p.start()
```

```

    for p in processes:
        p.join()

# Using Pool
def square(n):
    return n * n

if __name__ == "__main__":
    with Pool(4) as pool:
        results = pool.map(square, [1, 2, 3, 4, 5])
        print(results) # [1, 4, 9, 16, 25]

```

Concurrent Futures (Modern API)

```

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
import requests

# Threads for I/O
urls = ["http://example.com", "http://google.com", "http://github.com"]

def fetch(url):
    return len(requests.get(url).content)

with ThreadPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(fetch, urls))

# Processes for CPU
def cpu_heavy(n):
    return sum(i*i for i in range(n))

with ProcessPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(cpu_heavy, [10**6, 10**6, 10**6]))

```

Asyncio (Asynchronous I/O)

```

import asyncio

async def task(name, delay):
    print(f"Task {name} started")
    await asyncio.sleep(delay)
    print(f"Task {name} finished")
    return f"Result {name}"

async def main():
    # Run tasks concurrently
    results = await asyncio.gather(
        task("A", 2),
        task("B", 1),
        task("C", 3)
    )
    print(results)

asyncio.run(main())

```

Comparison

Approach	Best For	Memory	GIL
Threading	I/O-bound	Shared	Affected
Multiprocessing	CPU-bound	Separate	Not affected
Asyncio	I/O-bound (many)	Shared	Single-threaded

18. Python Standard Library Highlights

collections

```
from collections import Counter, defaultdict, OrderedDict, namedtuple, deque

# Counter - count occurrences
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
c = Counter(words)
print(c) # Counter({'apple': 3, 'banana': 2, 'cherry': 1})
print(c.most_common(2)) # [('apple', 3), ('banana', 2)]

# defaultdict - default values for missing keys
dd = defaultdict(list)
dd['a'].append(1)
dd['a'].append(2)
dd['b'].append(3)
# dd is now {'a': [1, 2], 'b': [3]}

# namedtuple - tuple with named fields
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y) # 10 20

# deque - double-ended queue (fast append/pop both ends)
d = deque([1, 2, 3])
d.append(4) # [1, 2, 3, 4]
d.appendleft(0) # [0, 1, 2, 3, 4]
d.pop() # [0, 1, 2, 3]
d.popleft() # [1, 2, 3]
```

datetime

```
from datetime import datetime, date, time, timedelta

# Current date and time
now = datetime.now()
today = date.today()

# Create dates
d = date(2023, 12, 25)
dt = datetime(2023, 12, 25, 10, 30, 0)

# Formatting
print(now.strftime("%Y-%m-%d %H:%M:%S")) # 2023-12-25 10:30:00
print(now.strftime("%B %d, %Y")) # December 25, 2023

# Parsing
dt = datetime.strptime("2023-12-25", "%Y-%m-%d")

# Arithmetic
tomorrow = today + timedelta(days=1)
next_week = today + timedelta(weeks=1)
diff = date(2024, 1, 1) - date(2023, 1, 1)
print(diff.days) # 365
```

os and sys

```
import os, sys

# Environment
print(os.environ.get('HOME'))
print(os.getcwd())
```

```

os.chdir('/path/to/dir')

# Paths
os.path.join('folder', 'subfolder', 'file.txt')
os.path.exists('/path')
os.path.isfile('/path')
os.path.isdir('/path')
os.path.basename('/a/b/c.txt') # c.txt
os.path.dirname('/a/b/c.txt') # /a/b
os.path.splitext('file.txt') # ('file', '.txt')

# Command-line arguments
print(sys.argv) # ['script.py', 'arg1', 'arg2']
print(sys.platform) # 'linux', 'win32', 'darwin'
print(sys.version)
sys.exit(0) # Exit program

```

random

```

import random

random.randint(1, 10) # Random int between 1-10
random.random() # Float between 0-1
random.uniform(1.0, 10.0) # Float between 1-10
random.choice(['a', 'b', 'c']) # Random element
random.sample([1,2,3,4,5], 3) # 3 unique random elements
random.shuffle(my_list) # Shuffle in place
random.seed(42) # For reproducibility

```

math

```

import math

math.pi # 3.141592...
math.e # 2.718281...
math.sqrt(16) # 4.0
math.pow(2, 10) # 1024.0
math.ceil(4.2) # 5
math.floor(4.8) # 4
math.factorial(5) # 120
math.gcd(12, 18) # 6
math.log(100, 10) # 2.0
math.sin(math.pi) # ~0

```

PART 2: IT CORE CONCEPTS

Essential computer science and IT fundamentals every software engineer should know.

19. Data Structures

Data structures are ways of organizing and storing data so it can be used efficiently. Choosing the right data structure is critical for writing efficient code.

Array

A collection of elements stored at contiguous memory locations, accessed by index. Fixed size (in many languages); Python's list is a dynamic array.

- **Access:** $O(1)$
- **Search:** $O(n)$
- **Insertion/Deletion:** $O(n)$ (worst case at beginning)

```
# Python list as array
arr = [10, 20, 30, 40, 50]
print(arr[2])           # 30 -  $O(1)$  access
arr.append(60)           #  $O(1)$  amortized
arr.insert(0, 5)         #  $O(n)$  - shift elements
arr.remove(30)           #  $O(n)$ 
```

Linked List

A linear collection of elements (nodes) where each node points to the next. Unlike arrays, elements are not stored in contiguous memory.

- **Access:** $O(n)$
- **Search:** $O(n)$
- **Insertion/Deletion at head:** $O(1)$

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def add_at_head(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def add_at_tail(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
```



```

ll = LinkedList()
ll.add_at_tail(1)
ll.add_at_tail(2)
ll.add_at_head(0)
ll.display() # 0 -> 1 -> 2 -> None

```

Types: Singly Linked List, Doubly Linked List, Circular Linked List.

Stack (LIFO)

Last-In-First-Out structure. Operations: push (add), pop (remove top), peek (top element).

- **All main operations:** $O(1)$
- **Use cases:** Function call stack, undo operations, expression evaluation, browser history

```

# Using list as stack
stack = []
stack.append(1) # push
stack.append(2)
stack.append(3)
print(stack.pop()) # 3 (pop top)
print(stack[-1]) # 2 (peek)

# Using collections.deque (preferred)
from collections import deque
stack = deque()
stack.append(1)
stack.append(2)
top = stack.pop()

```

Queue (FIFO)

First-In-First-Out structure. Operations: enqueue (add to rear), dequeue (remove from front).

- **All main operations:** $O(1)$
- **Use cases:** Task scheduling, breadth-first search, printer queues, asynchronous data transfer

```

from collections import deque

queue = deque()
queue.append(1) # enqueue
queue.append(2)
queue.append(3)
front = queue.popleft() # dequeue → 1

# Types:
# - Simple Queue (FIFO)
# - Circular Queue (last connects to first)
# - Priority Queue (elements with priority)
# - Deque (both ends)

# Priority Queue using heapq
import heapq
pq = []
heapq.heappush(pq, (2, "task2"))
heapq.heappush(pq, (1, "task1"))
heapq.heappush(pq, (3, "task3"))
print(heapq.heappop(pq)) # (1, 'task1')

```

Hash Table (Hash Map)

Maps keys to values using a hash function. Python's dict is a hash table.

- **Average Access/Insert/Delete:** $O(1)$

- **Worst case (collisions):** $O(n)$
- **Use cases:** Caches, database indexing, deduplication, frequency counting

```
# Python dict
hash_map = {}
hash_map["apple"] = 1      # O(1) insert
hash_map["banana"] = 2
print(hash_map["apple"])   # O(1) access
"apple" in hash_map        # O(1) lookup
del hash_map["banana"]     # O(1) delete

# Frequency count example
text = "hello world hello"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
# {'hello': 2, 'world': 1}
```

Tree

Hierarchical data structure with a root node and child nodes. No cycles.

Binary Tree

Each node has at most 2 children (left and right).

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# Build tree:
#       1
#      / \
#     2   3
#    / \
#   4   5

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)

# Tree Traversals
def inorder(node):    # Left -> Root -> Right
    if node:
        inorder(node.left)
        print(node.val, end=" ")
        inorder(node.right)

def preorder(node):  # Root -> Left -> Right
    if node:
        print(node.val, end=" ")
        preorder(node.left)
        preorder(node.right)

def postorder(node): # Left -> Right -> Root
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.val, end=" ")

inorder(root)        # 4 2 5 1 3
```

```
preorder(root)    # 1 2 4 5 3
postorder(root)   # 4 5 2 3 1
```

Binary Search Tree (BST)

A binary tree where for each node: left subtree values < node value < right subtree values. Allows $O(\log n)$ search.

- **Search/Insert/Delete (balanced):** $O(\log n)$
- **Worst case (skewed):** $O(n)$

```
class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        else:
            node.right = self._insert(node.right, val)
        return node

    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None or node.val == val:
            return node
        if val < node.val:
            return self._search(node.left, val)
        return self._search(node.right, val)
```

Heap

A complete binary tree satisfying the heap property: in a Max Heap, parent $\geq$ children; in a Min Heap, parent $\leq$ children.

- **Insert/Delete:** $O(\log n)$
- **Get min/max:** $O(1)$
- **Use cases:** Priority queues, heap sort, top-K problems

```
import heapq

# Min heap (default in Python)
heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 2)
print(heapq.heappop(heap))    # 1 (smallest)

# Max heap (negate values)
max_heap = []
heapq.heappush(max_heap, -3)
heapq.heappush(max_heap, -1)
print(-heapq.heappop(max_heap))  # 3 (largest)

# Find K smallest/largest
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
print(heapq.nsmallest(3, nums))  # [1, 1, 2]
```

```
print(heapq.nlargest(3, nums)) # [9, 6, 5]
```

Graph

A collection of nodes (vertices) connected by edges. Can be directed/undirected, weighted/unweighted.

- **Representations:** Adjacency Matrix, Adjacency List
- **Use cases:** Social networks, maps, network routing, recommendation systems

```
# Adjacency List representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

# Breadth-First Search (BFS)
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result

print(bfs(graph, 'A')) # ['A', 'B', 'C', 'D', 'E', 'F']

# Depth-First Search (DFS)
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=" ")
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

dfs(graph, 'A') # A B D E F C
```

Time Complexity Summary

Data Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)*	O(1)*
Stack	O(n)	O(n)	O(1)	O(1)
Queue	O(n)	O(n)	O(1)	O(1)
Hash Table	N/A	O(1) avg	O(1) avg	O(1) avg

BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)^{**}$	$O(n)$	$O(\log n)$	$O(\log n)$

* At known position. ** Only min/max element.

20. Algorithms & Complexity

Big O Notation

Big O describes the upper bound (worst case) of an algorithm's runtime or memory usage as input size grows.

Notation	Name	Example
$O(1)$	Constant	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Loop through array
$O(n \log n)$	Linearithmic	Merge sort, Quick sort (avg)
$O(n^2)$	Quadratic	Nested loops, Bubble sort
$O(n^3)$	Cubic	Triple nested loops
$O(2^n)$	Exponential	Recursive Fibonacci (naive)
$O(n!)$	Factorial	Permutations, traveling salesman

Order from best to worst: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Searching Algorithms

Linear Search - $O(n)$

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

print(linear_search([3, 1, 4, 1, 5, 9, 2, 6], 5)) # 4
```

Binary Search - $O(\log n)$

Requires sorted array. Repeatedly divides search space in half.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

sorted_arr = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(sorted_arr, 7)) # 3

# Recursive version
def binary_search_rec(arr, target, left, right):
    if left > right:
        return -1
```

```

mid = (left + right) // 2
if arr[mid] == target:
    return mid
elif arr[mid] < target:
    return binary_search_rec(arr, target, mid+1, right)
else:
    return binary_search_rec(arr, target, left, mid-1)

```

Sorting Algorithms

Bubble Sort - $O(n^2)$

Repeatedly compares adjacent elements and swaps them if in wrong order.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break # Already sorted
    return arr

print(bubble_sort([64, 34, 25, 12, 22, 11, 90]))

```

Selection Sort - $O(n^2)$

Finds the minimum in unsorted part and places it at the beginning.

```

def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

```

Insertion Sort - $O(n^2)$

Builds sorted array one element at a time by inserting each in its correct position. Efficient for nearly sorted small arrays.

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

```

Merge Sort - $O(n \log n)$

Divide-and-conquer: splits array into halves, sorts them, then merges.

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

```

```

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([5, 2, 8, 1, 9, 3])) # [1, 2, 3, 5, 8, 9]

```

Quick Sort - $O(n \log n)$ avg, $O(n^2)$ worst

Picks a pivot, partitions array around it, recursively sorts partitions.

```

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

print(quick_sort([10, 7, 8, 9, 1, 5]))

```

Sorting Comparison

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Recursion

A function calling itself with a smaller subproblem. Must have a base case to terminate.

```

# Factorial
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

```



```

# Fibonacci (naive -  $O(2^n)$ )
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

# Fibonacci with memoization ( $O(n)$ )
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

# Tower of Hanoi
def hanoi(n, source, target, helper):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n - 1, source, helper, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, helper, target, source)

```

Algorithm Paradigms

- **Brute Force:** Try all possibilities (e.g., linear search)
- **Divide and Conquer:** Split, solve sub-problems, combine (e.g., merge sort)
- **Greedy:** Make locally optimal choices (e.g., coin change with standard denominations)
- **Dynamic Programming:** Store results of subproblems (e.g., Fibonacci memoization)
- **Backtracking:** Try all paths, undo if not viable (e.g., N-Queens)

21. Database & SQL

Database Basics

A **database** is an organized collection of structured data. A **DBMS** (Database Management System) is software to manage databases (MySQL, PostgreSQL, Oracle, SQL Server).

Types of Databases

- **Relational (SQL)**: Data in tables with rows/columns. MySQL, PostgreSQL, Oracle.
- **NoSQL**: Non-tabular, flexible schema. MongoDB (document), Redis (key-value), Cassandra (column), Neo4j (graph).

Key Concepts

Term	Meaning
Table	Collection of related data (rows + columns)
Row/Record/Tuple	A single entry in a table
Column/Field/Attribute	A property of a record
Primary Key	Unique identifier for each row
Foreign Key	Column referencing another table's primary key
Schema	Structure/blueprint of the database
Index	Speeds up data retrieval
Constraint	Rule applied to data (NOT NULL, UNIQUE, etc.)

SQL Categories

Category	Full Form	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

Basic SQL Commands

Create and Modify Tables (DDL)

```
-- Create table
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10, 2),
  department VARCHAR(50),
```

```

        hire_date DATE DEFAULT CURRENT_DATE
    );

-- Alter table
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);
ALTER TABLE employees DROP COLUMN phone;
ALTER TABLE employees MODIFY salary DECIMAL(12, 2);

-- Drop table (delete completely)
DROP TABLE employees;

-- Truncate (remove all data, keep structure)
TRUNCATE TABLE employees;

```

Insert Data (DML)

```

-- Insert single row
INSERT INTO employees (name, email, salary, department)
VALUES ('Alice', 'alice@co.com', 50000, 'IT');

-- Insert multiple rows
INSERT INTO employees (name, salary, department) VALUES
    ('Bob', 60000, 'HR'),
    ('Carol', 70000, 'IT'),
    ('David', 55000, 'Sales');

```

Query Data (DQL - SELECT)

```

-- Select all
SELECT * FROM employees;

-- Select specific columns
SELECT name, salary FROM employees;

-- WHERE clause
SELECT * FROM employees WHERE department = 'IT';
SELECT * FROM employees WHERE salary > 50000;
SELECT * FROM employees WHERE name LIKE 'A%';      -- Starts with A
SELECT * FROM employees WHERE name LIKE '%son';    -- Ends with son
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 70000;
SELECT * FROM employees WHERE department IN ('IT', 'HR');
SELECT * FROM employees WHERE phone IS NULL;

-- Logical operators
SELECT * FROM employees
WHERE department = 'IT' AND salary > 60000;

SELECT * FROM employees
WHERE department = 'IT' OR department = 'HR';

-- ORDER BY
SELECT * FROM employees ORDER BY salary DESC;
SELECT * FROM employees ORDER BY department ASC, salary DESC;

-- LIMIT
SELECT * FROM employees LIMIT 5;
SELECT * FROM employees LIMIT 5 OFFSET 10;

-- DISTINCT
SELECT DISTINCT department FROM employees;

```

Update and Delete

```

-- Update
UPDATE employees
SET salary = 65000
WHERE id = 1;

```

```
-- Update multiple columns
UPDATE employees
SET salary = salary * 1.10, department = 'IT'
WHERE department = 'Tech';

-- Delete
DELETE FROM employees WHERE id = 5;
DELETE FROM employees WHERE department = 'Sales';

-- WARNING: DELETE without WHERE removes all rows!
-- DELETE FROM employees;
```

Aggregate Functions

```
SELECT COUNT(*) FROM employees;
SELECT SUM(salary) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MIN(salary), MAX(salary) FROM employees;

-- GROUP BY
SELECT department, COUNT(*) AS num_employees, AVG(salary) AS avg_sal
FROM employees
GROUP BY department;

-- HAVING (filter groups)
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > 55000;

-- Note: WHERE filters rows BEFORE grouping
--       HAVING filters groups AFTER grouping
```

Joins

Joins combine rows from multiple tables based on related columns.

```
-- Sample tables
-- employees(id, name, dept_id)
-- departments(id, dept_name, location)

-- INNER JOIN: only matching rows from both tables
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;

-- LEFT JOIN: all rows from left, matched from right (NULL if none)
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;

-- RIGHT JOIN: all rows from right, matched from left
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.id;

-- FULL OUTER JOIN: all rows from both tables
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.id;

-- CROSS JOIN: Cartesian product
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

```
-- SELF JOIN: join a table with itself
SELECT a.name AS employee, b.name AS manager
FROM employees a
INNER JOIN employees b ON a.manager_id = b.id;
```

Subqueries

```
-- Subquery in WHERE
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Subquery with IN
SELECT name FROM employees
WHERE dept_id IN (SELECT id FROM departments WHERE location = 'NYC');

-- Correlated subquery
SELECT e1.name, e1.salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Constraints

```
CREATE TABLE users (
    id INT PRIMARY KEY,           -- Unique + NOT NULL
    email VARCHAR(100) UNIQUE NOT NULL, -- Must be unique, not null
    age INT CHECK (age >= 18),    -- Constraint
    role VARCHAR(20) DEFAULT 'user', -- Default value
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(id)
);
```

Indexes

```
-- Create index (speeds up SELECT, slows INSERT/UPDATE)
CREATE INDEX idx_email ON employees(email);
CREATE UNIQUE INDEX idx_username ON users(username);

-- Composite index
CREATE INDEX idx_name_dept ON employees(name, dept_id);

-- Drop index
DROP INDEX idx_email ON employees;
```

Normalization

Process of organizing data to reduce redundancy and improve integrity.

- **1NF:** Atomic values (no repeating groups), unique rows
- **2NF:** 1NF + no partial dependencies on composite keys
- **3NF:** 2NF + no transitive dependencies (non-key fields don't depend on other non-key fields)
- **BCNF:** Stricter version of 3NF

ACID Properties

Properties of database transactions:

- **Atomicity:** All operations succeed or none do

- **Consistency:** Database moves from one valid state to another
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed changes persist (even after crash)

SQL vs NoSQL

Aspect	SQL	NoSQL
Schema	Fixed	Flexible
Scaling	Vertical	Horizontal
Data Model	Tables	Document/Key-Value/Graph
ACID	Strong	Eventual consistency (often)
Queries	SQL	Varies
Use Case	Complex queries, transactions	Big data, fast scaling

22. Operating Systems

What is an OS?

An **Operating System** is software that manages hardware and software resources and provides services for computer programs. Examples: Windows, macOS, Linux, Android, iOS.

Functions of OS

- Process management
- Memory management
- File system management
- Device management
- Security and access control
- User interface (CLI / GUI)
- Networking

Process vs Thread

Aspect	Process	Thread
Definition	An instance of a program	Smallest unit of execution within a process
Memory	Separate memory space	Shared memory within process
Creation	Heavy, slow	Lightweight, fast
Communication	IPC required	Direct via shared memory
Crash impact	Doesn't affect others	Can affect whole process

Process States

New → Ready → Running → Waiting/Blocked → Terminated

- **New:** Process is being created
- **Ready:** Waiting for CPU
- **Running:** Executing on CPU
- **Waiting:** Waiting for I/O or event
- **Terminated:** Finished execution

CPU Scheduling Algorithms

- **FCFS** (First Come First Served): Non-preemptive, simple, can have long wait times
- **SJF** (Shortest Job First): Optimal but needs prior knowledge of burst time
- **Priority Scheduling:** Process with highest priority runs; risk of starvation
- **Round Robin:** Each process gets a time quantum; good for time-sharing
- **Multilevel Queue:** Multiple queues with different priorities

Memory Management

Memory Hierarchy

- Registers (fastest, smallest)
- Cache (L1, L2, L3)
- Main Memory (RAM)
- Secondary Storage (SSD, HDD)
- Tertiary Storage (Tape, Cloud — slowest, largest)

Memory Allocation

- **Contiguous allocation:** Process gets a single contiguous block
- **Paging:** Memory divided into fixed-size pages
- **Segmentation:** Memory divided into variable-size segments

Virtual Memory

Allows programs to use more memory than physically available by swapping data between RAM and disk. **Page faults** occur when a page is not in RAM and must be fetched from disk.

Deadlock

A situation where two or more processes are waiting for each other to release resources, causing all to hang.

Four Necessary Conditions (Coffman conditions)

- **Mutual Exclusion:** Resources can't be shared
- **Hold and Wait:** Process holds a resource while waiting for others
- **No Preemption:** Resources can't be forcibly taken
- **Circular Wait:** Processes form a circular chain of dependencies

Deadlock Handling

- **Prevention:** Break one of the four conditions
- **Avoidance:** Algorithms like Banker's Algorithm
- **Detection & Recovery:** Detect cycle and kill process or release resources
- **Ignore:** Used in most OSes (Ostrich algorithm)

Synchronization

- **Critical Section:** Code that accesses shared resources
- **Mutex:** Lock allowing only one thread at a time
- **Semaphore:** Counter for limiting concurrent access
- **Race Condition:** Outcome depends on timing of threads

File Systems

- **Examples:** NTFS (Windows), ext4 (Linux), APFS (macOS), FAT32
- **File operations:** Create, read, write, delete, open, close, seek
- **Directory structure:** Single-level, two-level, tree, acyclic graph

23. Computer Networks

Network Basics

A **computer network** is a group of computers connected to share resources and communicate.

Types of Networks

- **LAN** (Local Area Network): Small area like home/office
- **MAN** (Metropolitan Area Network): City-wide
- **WAN** (Wide Area Network): Country/global (Internet)
- **PAN** (Personal Area Network): Bluetooth, USB

OSI Model (7 Layers)

Conceptual model for understanding network communication.

Layer	Name	Purpose	Examples
7	Application	User-facing protocols	HTTP, FTP, SMTP, DNS
6	Presentation	Data formatting, encryption	SSL/TLS, JPEG, ASCII
5	Session	Manage sessions	NetBIOS, RPC
4	Transport	End-to-end delivery	TCP, UDP
3	Network	Routing between networks	IP, ICMP, routers
2	Data Link	Node-to-node delivery	Ethernet, MAC, switches
1	Physical	Bits over physical medium	Cables, hubs, signals

TCP/IP Model (4 Layers)

Practical model used on the internet. Simplified compared to OSI.

- **Application** (OSI 5-7): HTTP, FTP, DNS, SMTP
- **Transport** (OSI 4): TCP, UDP
- **Internet** (OSI 3): IP, ICMP
- **Network Access** (OSI 1-2): Ethernet, Wi-Fi

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (acknowledgments)	Unreliable
Ordering	Maintains order	No order guarantee
Speed	Slower	Faster
Overhead	Higher	Lower

Use cases	Web, email, file transfer	Streaming, gaming, DNS
-----------	---------------------------	------------------------

IP Addressing

- **IPv4:** 32-bit (e.g., 192.168.1.1) — about 4.3 billion addresses
- **IPv6:** 128-bit (e.g., 2001:0db8:85a3::8a2e:0370:7334) — virtually unlimited
- **Private IPs:** 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- **Public IPs:** Routable on the internet

Common Protocols

Protocol	Port	Purpose
HTTP	80	Web (unencrypted)
HTTPS	443	Web (encrypted)
FTP	21	File transfer
SSH	22	Secure shell
SMTP	25/587	Sending email
POP3	110	Receiving email
IMAP	143	Email access
DNS	53	Domain name resolution
DHCP	67/68	Dynamic IP assignment
Telnet	23	Remote terminal (unsecure)

HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve data	Yes	Yes
POST	Submit new data	No	No
PUT	Update/replace resource	Yes	No
PATCH	Partial update	No*	No
DELETE	Remove resource	Yes	No
HEAD	Get headers only	Yes	Yes
OPTIONS	Get allowed methods	Yes	Yes

HTTP Status Codes

- **1xx Informational:** 100 Continue
- **2xx Success:** 200 OK, 201 Created, 204 No Content

- **3xx Redirection:** 301 Moved Permanently, 302 Found, 304 Not Modified
- **4xx Client Error:** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
- **5xx Server Error:** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

DNS (Domain Name System)

Translates human-readable domain names (e.g., google.com) into IP addresses (e.g., 142.250.190.46). Like a phonebook for the internet.

Network Devices

- **Router:** Routes packets between networks (Layer 3)
- **Switch:** Connects devices in a LAN (Layer 2)
- **Hub:** Basic device that broadcasts to all (Layer 1)
- **Modem:** Converts digital to analog (and vice versa)
- **Firewall:** Filters network traffic for security

Important Concepts

- **Bandwidth:** Maximum data transfer rate
- **Latency:** Time taken for data to travel
- **Throughput:** Actual data transferred per unit time
- **Packet:** Small unit of data sent over a network
- **MAC Address:** Hardware identifier of network card

24. Software Development Lifecycle (SDLC)

SDLC is the process of planning, creating, testing, and deploying software.

SDLC Phases

- **1. Requirement Analysis:** Gather and analyze user needs
- **2. Planning:** Define scope, resources, timeline
- **3. Design:** High-level (HLD) and low-level (LLD) design
- **4. Implementation/Coding:** Write the actual code
- **5. Testing:** Verify the software meets requirements
- **6. Deployment:** Release to production
- **7. Maintenance:** Fix bugs, add features, optimize

SDLC Models

Waterfall Model

Sequential — each phase completes before next begins. Good for stable requirements. Inflexible to changes.

Agile Model

Iterative and incremental. Delivers working software in short cycles (sprints, typically 2-4 weeks). Embraces change.

Agile Principles

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Scrum (Agile Framework)

- **Roles:** Product Owner, Scrum Master, Development Team
- **Artifacts:** Product Backlog, Sprint Backlog, Increment
- **Events:** Sprint Planning, Daily Standup, Sprint Review, Retrospective
- **Sprint:** Time-boxed iteration (usually 2-4 weeks)

Other Models

- **Spiral:** Iterative with risk analysis at each phase
- **V-Model:** Testing parallel to each development phase
- **DevOps:** Continuous integration, delivery, and deployment
- **Kanban:** Visual workflow with WIP limits

DevOps & CI/CD

- **DevOps:** Collaboration between Development and Operations
- **CI (Continuous Integration):** Automated build and test on every code commit

- **CD** (Continuous Delivery): Automated deployment to staging
- **Continuous Deployment:** Automated deployment to production
- **Tools:** Jenkins, GitHub Actions, GitLab CI, CircleCI, Docker, Kubernetes

25. Version Control (Git)

Version control tracks and manages changes to code. **Git** is the most popular distributed version control system.

Why Git?

- Track every change to the codebase
- Collaborate with multiple developers
- Revert to previous versions
- Branch for parallel development
- Distributed — full repo on every machine

Key Concepts

- **Repository (repo)**: A folder under version control
- **Commit**: A snapshot of changes with a message
- **Branch**: A parallel line of development
- **Merge**: Combine changes from different branches
- **Pull request (PR)**: Request to merge changes (GitHub/GitLab)
- **Working Directory** → **Staging Area** → **Local Repo** → **Remote Repo**

Essential Git Commands

```
# Configuration (one-time)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Initialize a new repo
git init

# Clone an existing repo
git clone https://github.com/user/repo.git

# Check status
git status

# Stage changes
git add filename.py          # Stage specific file
git add .                    # Stage all changes

# Commit changes
git commit -m "Add login feature"
git commit -am "Quick commit (skip stage)"

# View history
git log
git log --oneline            # Compact view
git log --graph --all        # Visual branch view

# Show changes
git diff                     # Unstaged changes
git diff --staged            # Staged changes

# Remote operations
git remote add origin <url>
```

```

git push origin main          # Push to remote
git pull origin main          # Pull from remote
git fetch origin              # Fetch without merging

# Branching
git branch                    # List branches
git branch new-feature        # Create branch
git checkout new-feature      # Switch branch
git checkout -b new-feature   # Create and switch
git switch new-feature        # Newer way to switch
git merge feature-branch      # Merge into current branch
git branch -d feature-branch  # Delete branch

# Undo
git checkout -- file.py       # Discard unstaged changes
git restore file.py           # Newer way to discard
git reset HEAD file.py        # Unstage
git reset --hard HEAD         # Discard everything
git revert <commit-hash>      # Create new commit undoing changes

# Stashing (save work temporarily)
git stash                     # Save changes
git stash pop                 # Apply and remove latest stash
git stash list                # List stashes

# Rebase (rewrite history – use carefully)
git rebase main               # Reapply commits on top of main

```

Branching Strategies

- **Git Flow:** main + develop + feature/release/hotfix branches
- **GitHub Flow:** main + feature branches with PRs
- **Trunk-Based:** Short-lived branches merged frequently to main

Common Workflow

```

# 1. Clone repo
git clone <repo-url>
cd repo

# 2. Create feature branch
git checkout -b feature/new-login

# 3. Make changes, then commit
git add .
git commit -m "Add login form"

# 4. Push to remote
git push origin feature/new-login

# 5. Create Pull Request on GitHub/GitLab
# 6. After review and merge, update local main
git checkout main
git pull origin main
git branch -d feature/new-login

```

26. Web Technologies & REST APIs

How the Web Works

Client (browser) sends an HTTP request to a server. Server processes and sends back an HTTP response with HTML/JSON/etc.

- **Client:** Web browser, mobile app, etc.
- **Server:** Computer hosting the website/API
- **Frontend:** What user sees (HTML, CSS, JS)
- **Backend:** Server logic, database (Python, Java, Node.js)

Web Technologies

- **HTML:** Structure of web pages
- **CSS:** Styling and layout
- **JavaScript:** Interactivity in browsers
- **Frontend Frameworks:** React, Angular, Vue.js
- **Backend Frameworks:** Django, Flask, FastAPI (Python); Spring (Java); Express (Node.js)

Client-Server Architecture

- **Two-tier:** Client ↔ Server
- **Three-tier:** Client ↔ Application Server ↔ Database
- **Microservices:** Multiple small independent services
- **Monolithic:** Single large application

REST API

REST (Representational State Transfer) is an architectural style for building APIs over HTTP. RESTful APIs use HTTP methods to perform CRUD operations on resources.

REST Principles

- **Stateless:** Each request contains all info needed
- **Client-Server:** Separation of concerns
- **Cacheable:** Responses can be cached
- **Uniform Interface:** Consistent resource URIs
- **Layered System:** Multiple layers (load balancer, server, DB)

CRUD Operations Mapping

Operation	HTTP Method	Endpoint Example	Description
Create	POST	/users	Create new user
Read (all)	GET	/users	Get all users
Read (one)	GET	/users/123	Get user with id 123
Update (full)	PUT	/users/123	Replace user 123

Update (partial)	PATCH	/users/123	Update user 123 fields
Delete	DELETE	/users/123	Delete user 123

Example REST API Request/Response

```
# Request
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer <token>

{
  "name": "Alice",
  "email": "alice@example.com"
}

# Response
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 123,
  "name": "Alice",
  "email": "alice@example.com",
  "created_at": "2024-01-15T10:30:00Z"
}
```

Building a Simple REST API in Python (Flask)

```
from flask import Flask, jsonify, request

app = Flask(__name__)
users = []

# Get all users
@app.route('/users', methods=['GET'])
def get_users():
    return jsonify(users)

# Get one user
@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = next((u for u in users if u['id'] == user_id), None)
    if user:
        return jsonify(user)
    return jsonify({"error": "Not found"}), 404

# Create user
@app.route('/users', methods=['POST'])
def create_user():
    data = request.get_json()
    user = {"id": len(users) + 1, **data}
    users.append(user)
    return jsonify(user), 201

# Update user
@app.route('/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    data = request.get_json()
    for user in users:
        if user['id'] == user_id:
            user.update(data)
            return jsonify(user)
    return jsonify({"error": "Not found"}), 404
```

```
# Delete user
@app.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    global users
    users = [u for u in users if u['id'] != user_id]
    return '', 204

if __name__ == '__main__':
    app.run(debug=True)
```

Consuming APIs in Python

```
import requests

# GET request
response = requests.get('https://api.example.com/users')
print(response.status_code)    # 200
print(response.json())         # JSON data

# POST request
data = {"name": "Alice", "email": "a@a.com"}
response = requests.post(
    'https://api.example.com/users',
    json=data,
    headers={"Authorization": "Bearer token123"}
)

# Handle response
if response.status_code == 201:
    print("Created:", response.json())
else:
    print("Error:", response.text)

# Query parameters
params = {"page": 1, "limit": 10}
response = requests.get('https://api.example.com/users', params=params)
```

API Authentication

- **API Key:** Token passed in header or query string
- **Basic Auth:** Username and password (base64-encoded)
- **Bearer Token / JWT:** Token in Authorization header
- **OAuth 2.0:** Industry-standard for delegated access

Other API Styles

- **SOAP:** XML-based, more rigid, used in enterprise
- **GraphQL:** Query language, client specifies exact data needed
- **gRPC:** Google's high-performance RPC, uses Protocol Buffers
- **WebSocket:** Bidirectional persistent connection (chat, gaming)

27. Cloud Computing Basics

What is Cloud Computing?

Delivery of computing services (servers, storage, databases, networking, software) over the internet ('the cloud'). Pay-as-you-go model.

Service Models

Model	Full Form	You Manage	Provider Manages	Examples
IaaS	Infrastructure as a Service	OS, runtime, data, apps	Hardware, virtualization	AWS EC2, Azure VMs
PaaS	Platform as a Service	Data, apps	Everything else	Heroku, App Engine
SaaS	Software as a Service	Just use it	Everything	Gmail, Salesforce
FaaS	Function as a Service	Code only	Everything else	AWS Lambda, GCP Functions

Deployment Models

- **Public Cloud:** Owned by provider, shared by many (AWS, Azure, GCP)
- **Private Cloud:** Dedicated to one organization
- **Hybrid Cloud:** Mix of public and private
- **Multi-Cloud:** Using multiple cloud providers

Major Cloud Providers

- **AWS** (Amazon Web Services): Market leader. Services: EC2, S3, RDS, Lambda, etc.
- **Microsoft Azure:** Strong in enterprise. Services: VMs, Blob Storage, SQL Database.
- **Google Cloud Platform (GCP):** Strong in AI/ML, data. Services: Compute Engine, BigQuery.

Key Cloud Concepts

- **Scalability:** Increase resources to handle load
 - - Vertical scaling (scale up): bigger machine
 - - Horizontal scaling (scale out): more machines
- **High Availability:** System remains operational
- **Fault Tolerance:** Continues working despite failures
- **Load Balancer:** Distributes traffic across servers
- **Auto-scaling:** Automatically adjusts resources
- **CDN** (Content Delivery Network): Distributed servers serving cached content

Containers & Orchestration

- **Docker:** Containers — lightweight, portable software packages
- **Kubernetes:** Orchestrates containers (deployment, scaling, management)
- **Container Registry:** Docker Hub, AWS ECR — store container images

28. Software Testing

Testing verifies that software works correctly and meets requirements.

Types of Testing

By Knowledge of Internal Code

- **Black Box Testing:** Testing without knowledge of internal code (functional testing)
- **White Box Testing:** Testing with knowledge of internal code (structural testing)
- **Gray Box Testing:** Partial knowledge

By Level

- **Unit Testing:** Test individual functions/methods (smallest unit)
- **Integration Testing:** Test how modules work together
- **System Testing:** Test complete application end-to-end
- **Acceptance Testing:** Test against business requirements (UAT)

By Execution Method

- **Manual Testing:** Performed by humans
- **Automated Testing:** Performed by scripts/tools

Other Types

- **Regression Testing:** Ensure new changes don't break existing functionality
- **Smoke Testing:** Basic checks to ensure build is stable
- **Sanity Testing:** Quick check after minor changes
- **Performance Testing:** Speed, responsiveness, stability
- **Load Testing:** Behavior under expected load
- **Stress Testing:** Behavior under extreme conditions
- **Security Testing:** Find vulnerabilities
- **Usability Testing:** User-friendliness

Unit Testing in Python

```
import unittest

# Function to test
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

# Test class
class TestMathOperations(unittest.TestCase):

    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
```

```

        self.assertEqual(add(-1, -1), -2)

    def test_divide(self):
        self.assertEqual(divide(10, 2), 5)

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            divide(10, 0)

    def setUp(self):
        # Runs before each test
        pass

    def tearDown(self):
        # Runs after each test
        pass

if __name__ == '__main__':
    unittest.main()

# Run: python -m unittest test_file.py

```

Common Assertions

<code>self.assertEqual(a, b)</code>	<code># a == b</code>
<code>self.assertNotEqual(a, b)</code>	<code># a != b</code>
<code>self.assertTrue(x)</code>	<code># x is truthy</code>
<code>self.assertFalse(x)</code>	<code># x is falsy</code>
<code>self.assertIsNone(x)</code>	<code># x is None</code>
<code>self.assertIsNotNone(x)</code>	<code># x is not None</code>
<code>self.assertIn(a, b)</code>	<code># a in b</code>
<code>self.assertNotIn(a, b)</code>	<code># a not in b</code>
<code>self.assertIsInstance(a, T)</code>	<code># isinstance(a, T)</code>
<code>self.assertRaises(Exception)</code>	<code># raises Exception</code>
<code>self.assertAlmostEqual(a, b)</code>	<code># for floats</code>

pytest (Popular Testing Framework)

```

# test_math.py
import pytest

def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_add_strings():
    assert add("hello", " world") == "hello world"

# Parametrize tests
@pytest.mark.parametrize("a, b, expected", [
    (1, 2, 3),
    (5, 5, 10),
    (-1, 1, 0),
])
def test_add_param(a, b, expected):
    assert add(a, b) == expected

# Run: pytest test_math.py

```

Test-Driven Development (TDD)

Approach where you write tests **before** code:

- **1. Red:** Write a failing test
- **2. Green:** Write minimum code to make it pass
- **3. Refactor:** Improve code while keeping tests passing

PART 3: INTERVIEW PREPARATION

This section contains carefully curated interview questions with complete solutions, organized by difficulty level (L1 and L2), plus theory questions, HR/behavioral questions, and practical tips for cracking your associate software engineer interview.

29. L1 Coding Interview Questions (with Solutions)

L1 questions test your fundamental programming skills. They focus on basic logic, loops, conditionals, and simple data structure manipulation. Most can be solved in 5-15 minutes.

Q1. Reverse a String

```
# Method 1: Using slicing (Pythonic)
def reverse_string(s):
    return s[::-1]

# Method 2: Using loop
def reverse_string_loop(s):
    result = ""
    for char in s:
        result = char + result
    return result

# Method 3: Using reversed()
def reverse_string_reversed(s):
    return "".join(reversed(s))

print(reverse_string("hello")) # olleh
```

Q2. Check if a String is a Palindrome

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

# Two-pointer approach (more efficient)
def is_palindrome_v2(s):
    s = s.lower().replace(" ", "")
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("racecar")) # True
print(is_palindrome("hello")) # False
print(is_palindrome("A man a plan a canal Panama")) # True
```

Q3. Fibonacci Series

```
# Method 1: Iterative (best for interviews)
def fibonacci(n):
    a, b = 0, 1
    result = []
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result

# Method 2: Recursive (inefficient - O(2^n))
def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)

# Method 3: Recursive with memoization (efficient)
```



```
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

print(fibonacci(10)) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Q4. Factorial of a Number

```
# Iterative
def factorial(n):
    if n < 0:
        return None
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Recursive
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

# Using math module
import math
print(math.factorial(5)) # 120
print(factorial(5))      # 120
```

Q5. Check if a Number is Prime

```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    # Only check up to sqrt(n)
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

print(is_prime(17)) # True
print(is_prime(20)) # False
print(is_prime(2))  # True
```

Q6. Find Largest and Smallest in a List

```
def find_min_max(arr):
    if not arr:
        return None, None
    min_val = max_val = arr[0]
    for num in arr[1:]:
        if num < min_val:
            min_val = num
        elif num > max_val:
            max_val = num
    return min_val, max_val
```

```
# Built-in
arr = [3, 1, 4, 1, 5, 9, 2, 6]
print(min(arr), max(arr)) # 1 9
print(find_min_max(arr)) # (1, 9)
```

Q7. Sum of Digits

```
def sum_of_digits(n):
    total = 0
    n = abs(n)
    while n > 0:
        total += n % 10
        n //= 10
    return total

# One-liner
def sum_digits_v2(n):
    return sum(int(d) for d in str(abs(n)))

print(sum_of_digits(12345)) # 15
print(sum_digits_v2(12345)) # 15
```

Q8. Count Vowels and Consonants

```
def count_vowels_consonants(s):
    vowels = "aeiouAEIOU"
    v_count = c_count = 0
    for char in s:
        if char.isalpha():
            if char in vowels:
                v_count += 1
            else:
                c_count += 1
    return v_count, c_count

v, c = count_vowels_consonants("Hello World")
print(f"Vowels: {v}, Consonants: {c}") # Vowels: 3, Consonants: 7
```

Q9. Swap Two Variables Without a Temp Variable

```
# Pythonic way
a, b = 5, 10
a, b = b, a
print(a, b) # 10 5

# Using arithmetic
a, b = 5, 10
a = a + b # a = 15
b = a - b # b = 5
a = a - b # a = 10
print(a, b) # 10 5

# Using XOR (for integers)
a, b = 5, 10
a = a ^ b
b = a ^ b
a = a ^ b
print(a, b) # 10 5
```

Q10. FizzBuzz

Classic interview question! Print numbers 1-100. For multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', multiples of both print 'FizzBuzz'.

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)

fizzbuzz(15)
# 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz,
# 11, Fizz, 13, 14, FizzBuzz
```

Q11. Find Duplicates in a List

```
def find_duplicates(arr):
    seen = set()
    duplicates = set()
    for item in arr:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

# Using Counter
from collections import Counter
def find_duplicates_v2(arr):
    count = Counter(arr)
    return [item for item, freq in count.items() if freq > 1]

print(find_duplicates([1, 2, 3, 2, 4, 5, 3])) # [2, 3]
```

Q12. Count Occurrences of a Character

```
def count_char(s, ch):
    return s.count(ch)

# Manual count
def count_char_manual(s, ch):
    count = 0
    for c in s:
        if c == ch:
            count += 1
    return count

# Count all characters
from collections import Counter
print(Counter("hello")) # Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
```

Q13. Reverse a Number

```
def reverse_number(n):
    is_negative = n < 0
    n = abs(n)
    reversed_n = 0
    while n > 0:
        digit = n % 10
        reversed_n = reversed_n * 10 + digit
        n //= 10
    return -reversed_n if is_negative else reversed_n
```

```
print(reverse_number(12345)) # 54321
print(reverse_number(-678)) # -876
```

Q14. Check Armstrong Number

An Armstrong number is one where the sum of its digits raised to the power of the number of digits equals the number itself. E.g., $153 = 1^3 + 5^3 + 3^3$

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    total = sum(int(d) ** power for d in digits)
    return total == n

print(is_armstrong(153)) # True (1+125+27=153)
print(is_armstrong(9474)) # True
print(is_armstrong(123)) # False
```

Q15. Sum of Natural Numbers

```
# Using loop
def sum_natural(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

# Using formula (O(1))
def sum_natural_formula(n):
    return n * (n + 1) // 2

# Using built-in
print(sum(range(1, 101))) # 5050
print(sum_natural_formula(100)) # 5050
```

Q16. Generate Multiplication Table

```
def multiplication_table(n, limit=10):
    for i in range(1, limit + 1):
        print(f"{n} x {i} = {n * i}")

multiplication_table(5)
# 5 x 1 = 5
# 5 x 2 = 10
# ...
# 5 x 10 = 50
```

Q17. Check Even or Odd

```
def check_even_odd(n):
    return "Even" if n % 2 == 0 else "Odd"

# Using bitwise (faster)
def check_even_odd_bitwise(n):
    return "Odd" if n & 1 else "Even"

print(check_even_odd(7)) # Odd
print(check_even_odd(10)) # Even
```

Q18. Find GCD and LCM

```
def gcd(a, b):
    while b:
```

```

        a, b = b, a % b
    return a

def lcm(a, b):
    return (a * b) // gcd(a, b)

# Using math module
import math
print(math.gcd(12, 18)) # 6
print(math.lcm(4, 6))   # 12 (Python 3.9+)

```

Q19. Count Words in a Sentence

```

def count_words(sentence):
    return len(sentence.split())

# Count word frequency
from collections import Counter
def word_frequency(sentence):
    return Counter(sentence.lower().split())

print(count_words("Hello World Python")) # 3
print(word_frequency("the cat sat on the mat"))
# Counter({'the': 2, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1})

```

Q20. Remove Duplicates from a List

```

# Method 1: Using set (loses order)
def remove_duplicates(arr):
    return list(set(arr))

# Method 2: Preserve order (Python 3.7+)
def remove_duplicates_ordered(arr):
    return list(dict.fromkeys(arr))

# Method 3: Manual
def remove_duplicates_manual(arr):
    seen = set()
    result = []
    for item in arr:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_duplicates_ordered([1, 2, 2, 3, 4, 4, 5])) # [1, 2, 3, 4, 5]

```

30. L2 Coding Interview Questions (with Solutions)

L2 questions require deeper algorithmic thinking, optimization, and knowledge of data structures. They are common in second-round technical interviews.

Q1. Two Sum

Given an array of integers and a target sum, return the indices of two numbers that add up to the target.

```
# Brute force O(n^2)
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

# Optimized using hash map O(n)
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
print(two_sum([3, 2, 4], 6))     # [1, 2]
```

Q2. Check if Two Strings are Anagrams

```
# Method 1: Sorting O(n log n)
def is_anagram_sort(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())

# Method 2: Counter O(n)
from collections import Counter
def is_anagram(s1, s2):
    return Counter(s1.lower()) == Counter(s2.lower())

# Method 3: Manual count O(n)
def is_anagram_manual(s1, s2):
    if len(s1) != len(s2):
        return False
    count = {}
    for char in s1.lower():
        count[char] = count.get(char, 0) + 1
    for char in s2.lower():
        if char not in count or count[char] == 0:
            return False
        count[char] -= 1
    return True

print(is_anagram("listen", "silent")) # True
print(is_anagram("hello", "world"))   # False
```

Q3. Find Missing Number in Array

Given an array of n distinct numbers in range [0, n], find the missing number.

```
# Method 1: Sum formula O(n)
def missing_number(nums):
```

```

n = len(nums)
expected_sum = n * (n + 1) // 2
return expected_sum - sum(nums)

# Method 2: XOR O(n) - more elegant
def missing_number_xor(nums):
    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i ^ num
    return result

# Method 3: Set difference
def missing_number_set(nums):
    return (set(range(len(nums) + 1)) - set(nums)).pop()

print(missing_number([3, 0, 1])) # 2
print(missing_number([9, 6, 4, 2, 3, 5, 7, 0, 1])) # 8

```

Q4. Rotate an Array by K Positions

```

# Method 1: Using slicing (extra space)
def rotate_array(arr, k):
    k = k % len(arr)
    return arr[-k:] + arr[:-k]

# Method 2: In-place using reversal O(n) time, O(1) space
def rotate_in_place(arr, k):
    n = len(arr)
    k = k % n
    # Reverse entire array
    arr.reverse()
    # Reverse first k elements
    arr[:k] = arr[:k][::-1]
    # Reverse remaining elements
    arr[k:] = arr[k:][::-1]
    return arr

print(rotate_array([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]
print(rotate_in_place([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]

```

Q5. Valid Parentheses

Given a string of brackets, determine if it is valid (every open bracket has matching close bracket of same type, in correct order).

```

def is_valid_parentheses(s):
    stack = []
    mapping = {'(': ')', '[': ']', '{': '}'

    for char in s:
        if char in mapping: # Closing bracket
            if not stack or stack.pop() != mapping[char]:
                return False
        else: # Opening bracket
            stack.append(char)

    return len(stack) == 0

print(is_valid_parentheses("()")) # True
print(is_valid_parentheses("()[{}]") # True
print(is_valid_parentheses("]")) # False
print(is_valid_parentheses("([])") # False
print(is_valid_parentheses("{[]}")) # True

```

Q6. Merge Two Sorted Arrays

```
def merge_sorted(arr1, arr2):
    result = []
    i = j = 0
    while i < len(arr1) and j < len(arr2):
        if arr1[i] <= arr2[j]:
            result.append(arr1[i])
            i += 1
        else:
            result.append(arr2[j])
            j += 1
    # Append remaining elements
    result.extend(arr1[i:])
    result.extend(arr2[j:])
    return result

print(merge_sorted([1, 3, 5], [2, 4, 6])) # [1, 2, 3, 4, 5, 6]
print(merge_sorted([1, 2, 3], [4, 5, 6])) # [1, 2, 3, 4, 5, 6]
```

Q7. Reverse a Linked List

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Iterative O(n) time, O(1) space
def reverse_list(head):
    prev = None
    current = head
    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node
    return prev

# Recursive
def reverse_list_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverse_list_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head

# Helper to print
def print_list(head):
    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")
```

Q8. Find Maximum Subarray Sum (Kadane's Algorithm)

Find the contiguous subarray with the largest sum.

```
def max_subarray(nums):
    max_sum = current_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum
```



```
# With subarray indices
def max_subarray_with_indices(nums):
    max_sum = current_sum = nums[0]
    start = end = temp_start = 0
    for i in range(1, len(nums)):
        if nums[i] > current_sum + nums[i]:
            current_sum = nums[i]
            temp_start = i
        else:
            current_sum += nums[i]
        if current_sum > max_sum:
            max_sum = current_sum
            start = temp_start
            end = i
    return max_sum, nums[start:end+1]

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
# 6 (subarray: [4, -1, 2, 1])
```

Q9. Longest Substring Without Repeating Characters

```
def longest_unique_substring(s):
    char_index = {}
    max_length = 0
    start = 0

    for i, char in enumerate(s):
        if char in char_index and char_index[char] >= start:
            start = char_index[char] + 1
        char_index[char] = i
        max_length = max(max_length, i - start + 1)

    return max_length

print(longest_unique_substring("abcabcbb")) # 3 ("abc")
print(longest_unique_substring("bbbbbb")) # 1 ("b")
print(longest_unique_substring("pwwkew")) # 3 ("wke")
```

Q10. Binary Search

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Find first occurrence (for duplicates)
def find_first(arr, target):
    left, right = 0, len(arr) - 1
    result = -1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            result = mid
            right = mid - 1 # Continue searching left
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
```

```

        return result

print(binary_search([1, 3, 5, 7, 9, 11], 7)) # 3

```

Q11. Find Duplicate Number

Find the duplicate in an array of $n+1$ integers where each integer is between 1 and n .

```

# Method 1: Hash set  $O(n)$  time,  $O(n)$  space
def find_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)
    return -1

# Method 2: Floyd's Cycle Detection  $O(n)$  time,  $O(1)$  space
def find_duplicate_floyd(nums):
    slow = fast = nums[0]
    # Phase 1: Find meeting point
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    # Phase 2: Find entry of cycle
    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]
    return slow

print(find_duplicate([1, 3, 4, 2, 2])) # 2
print(find_duplicate([3, 1, 3, 4, 2])) # 3

```

Q12. Climbing Stairs (Dynamic Programming)

You can climb 1 or 2 steps at a time. How many distinct ways to reach the top of n stairs?

```

# Recursive (exponential)
def climb_stairs_recursive(n):
    if n <= 2:
        return n
    return climb_stairs_recursive(n-1) + climb_stairs_recursive(n-2)

# DP -  $O(n)$  time,  $O(n)$  space
def climb_stairs_dp(n):
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Optimized -  $O(n)$  time,  $O(1)$  space
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current
    return prev1

```

```
print(climb_stairs(5)) # 8
```

Q13. Best Time to Buy and Sell Stock

Given an array of prices, find max profit from one buy-sell transaction.

```
def max_profit(prices):
    if not prices:
        return 0
    min_price = prices[0]
    max_profit = 0
    for price in prices[1:]:
        if price < min_price:
            min_price = price
        elif price - min_price > max_profit:
            max_profit = price - min_price
    return max_profit

print(max_profit([7, 1, 5, 3, 6, 4])) # 5 (buy at 1, sell at 6)
print(max_profit([7, 6, 4, 3, 1])) # 0 (no profit)
```

Q14. Find Intersection of Two Arrays

```
# Using sets
def intersection(arr1, arr2):
    return list(set(arr1) & set(arr2))

# With counts (Counter)
from collections import Counter
def intersect_with_counts(arr1, arr2):
    c1 = Counter(arr1)
    c2 = Counter(arr2)
    result = []
    for num in c1:
        if num in c2:
            result.extend([num] * min(c1[num], c2[num]))
    return result

print(intersection([1, 2, 2, 1], [2, 2])) # [2]
print(intersect_with_counts([1, 2, 2, 1], [2, 2])) # [2, 2]
```

Q15. Group Anagrams

```
from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        # Sorted string is the key
        key = "".join(sorted(s))
        groups[key].append(s)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Q16. Implement Queue using Stacks

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x):
```

```

        self.in_stack.append(x)

    def pop(self):
        self._move()
        return self.out_stack.pop()

    def peek(self):
        self._move()
        return self.out_stack[-1]

    def empty(self):
        return not self.in_stack and not self.out_stack

    def _move(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())

q = MyQueue()
q.push(1); q.push(2); q.push(3)
print(q.pop())    # 1
print(q.peek())   # 2

```

Q17. Find First Non-Repeating Character

```

from collections import Counter

def first_non_repeating(s):
    count = Counter(s)
    for i, char in enumerate(s):
        if count[char] == 1:
            return i
    return -1

print(first_non_repeating("leetcode"))    # 0 (l)
print(first_non_repeating("loveleetcode")) # 2 (v)
print(first_non_repeating("aabb"))        # -1

```

Q18. Power of Two

```

# Method 1: Bit manipulation
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

# Method 2: Loop
def is_power_of_two_loop(n):
    if n <= 0:
        return False
    while n % 2 == 0:
        n //= 2
    return n == 1

print(is_power_of_two(16)) # True
print(is_power_of_two(18)) # False
print(is_power_of_two(1))  # True

```

Q19. Maximum Depth of Binary Tree

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

```

# Recursive DFS
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

# Iterative BFS
from collections import deque
def max_depth_bfs(root):
    if not root:
        return 0
    queue = deque([root])
    depth = 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return depth

```

Q20. Move Zeros to End

```

# In-place, preserves order O(n) time, O(1) space
def move_zeros(arr):
    pos = 0 # Position to place next non-zero
    for i in range(len(arr)):
        if arr[i] != 0:
            arr[pos], arr[i] = arr[i], arr[pos]
            pos += 1
    return arr

print(move_zeros([0, 1, 0, 3, 12])) # [1, 3, 12, 0, 0]
print(move_zeros([0, 0, 1]))       # [1, 0, 0]

```

31. Theory & Conceptual Questions

These questions test your conceptual understanding. Be prepared to explain clearly with examples.

Python-Specific Questions

O1. What is the difference between a List and a Tuple?

Note: Answer: Lists are mutable (can be modified after creation), use square brackets [], and are slower. Tuples are immutable (cannot be changed), use parentheses (), are faster, and can be used as dictionary keys. Use tuples for fixed data and lists for dynamic collections.

O2. What is the difference between == and is?

Note: Answer: '==' compares the VALUES of two objects (equality), while 'is' compares the IDENTITY (memory location) of two objects. Example: a = [1,2,3]; b = [1,2,3]; a == b is True, but a is b is False because they're different objects in memory.

O3. What are mutable and immutable types in Python?

Note: Answer: Mutable types can be changed after creation: list, dict, set. Immutable types cannot be changed: int, float, string, tuple, frozenset, bool. When you 'modify' an immutable, you actually create a new object.

O4. What is the GIL (Global Interpreter Lock)?

Note: Answer: The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This means true parallelism is not possible with threads for CPU-bound tasks. Use multiprocessing for CPU-bound work, threading for I/O-bound work.

O5. What is the difference between deep copy and shallow copy?

Note: Answer: Shallow copy (copy.copy()) creates a new object but inserts references to objects found in the original. Deep copy (copy.deepcopy()) creates a new object AND recursively copies all objects inside it. For nested structures, deep copy is needed for full independence.

O6. What are *args and **kwargs?

Note: Answer: *args allows passing variable number of positional arguments (received as tuple). **kwargs allows passing variable number of keyword arguments (received as dict). Order: def f(positional, *args, default=val, **kwargs)

O7. What is the difference between a generator and an iterator?

Note: Answer: An iterator is any object implementing __iter__() and __next__(). A generator is a special type of iterator created using a function with 'yield'. Generators are simpler to write, more memory-efficient (lazy evaluation), and produce values on-demand.

O8. What are decorators?

Note: Answer: Decorators are functions that take another function as input and extend its behavior without explicitly modifying it. They use the @decorator syntax and are commonly used for logging, authentication, timing, caching, etc.

O9. Explain Python's memory management.

Note: Answer: Python uses automatic memory management with reference counting (each object has a reference count; when it reaches 0, memory is freed) and a generational garbage collector for cyclic references. Memory is managed by the Python Memory Manager.

O10. What is the difference between range() and xrange()?

Note: Answer: In Python 2, range() returns a list while xrange() returns an iterator. In Python 3, xrange() is removed; range() behaves like xrange() (returns a range object that produces values on demand).

O11. What is list comprehension?

Note: Answer: A concise way to create lists. Syntax: [expression for item in iterable if condition]. Example: [x*2 for x in range(5) if x%2==0] → [0, 4, 8]. Faster than equivalent for-loops.

O12. What are dunder/magic methods?

Note: Answer: Special methods surrounded by double underscores (e.g., __init__, __str__, __len__, __add__). They enable operator overloading and customize object behavior. __init__ is the constructor, __str__ returns string representation, __repr__ returns detailed representation.

O13. What is the difference between @staticmethod and @classmethod?

Note: Answer: @staticmethod doesn't receive 'self' or 'cls'; it's a regular function placed in a class. @classmethod receives 'cls' (the class) as first argument and can access/modify class state. Use classmethod for alternative constructors.

O14. What is a Lambda function?

Note: Answer: An anonymous, single-expression function. Syntax: lambda args: expression. Example: square = lambda x: x**2. Useful for short functions, especially with map(), filter(), sorted().

O15. How does Python handle exceptions?

Note: Answer: Using try-except-else-finally. 'try' contains risky code, 'except' handles specific exceptions, 'else' runs if no exception occurred, 'finally' always runs (for cleanup). You can raise exceptions with 'raise' and create custom exceptions by inheriting from Exception.

OOP Questions

O16. What are the four pillars of OOP?

Note: Answer: (1) Encapsulation - bundling data and methods, hiding internal details. (2) Inheritance - deriving new classes from existing ones. (3) Polymorphism - same interface, different implementations. (4) Abstraction - hiding complexity, showing only essential features.

O17. What is the difference between method overloading and overriding?

Note: Answer: Overloading: multiple methods with same name but different parameters (Python doesn't support traditional overloading; uses default args or *args). Overriding: subclass provides specific implementation of a method already in parent class.

O18. What is multiple inheritance and the MRO?

Note: Answer: Multiple inheritance allows a class to inherit from multiple parent classes. Python uses MRO (Method Resolution Order) — specifically C3 linearization — to determine which method to call. Check it with ClassName.__mro__ or ClassName.mro().

O19. What is an abstract class?

Note: Answer: A class that cannot be instantiated and may contain abstract methods (methods without implementation). In Python, use ABC module: 'from abc import ABC, abstractmethod'. Subclasses must implement all abstract methods.

Q20. What's the difference between composition and inheritance?

Note: Answer: Inheritance is 'is-a' relationship (Dog is an Animal). Composition is 'has-a' relationship (Car has an Engine). Composition is generally preferred as it provides more flexibility and less tight coupling. 'Favor composition over inheritance' is a common design principle.

Data Structures & Algorithms

Q21. What is the time complexity of common operations?

Operation	List	Dict/Set	Tuple
Access by index	$O(1)$	N/A	$O(1)$
Search	$O(n)$	$O(1)$	$O(n)$
Insert/Append	$O(1)$ end, $O(n)$ middle	$O(1)$	Immutable
Delete	$O(n)$	$O(1)$	Immutable
Length len()	$O(1)$	$O(1)$	$O(1)$

Q22. Stack vs Queue?

Note: Answer: Stack is LIFO (Last In, First Out) — like a stack of plates. Operations: push (add to top), pop (remove from top). Queue is FIFO (First In, First Out) — like a line at a store. Operations: enqueue (add to rear), dequeue (remove from front).

Q23. What is recursion? What are its drawbacks?

Note: Answer: Recursion is when a function calls itself with a smaller subproblem. Needs a base case to stop. Drawbacks: high memory usage (call stack), can be slower than iteration, risk of stack overflow for deep recursion. Use when problem has clear recursive structure (trees, divide-and-conquer).

Q24. Binary Search Tree (BST) properties?

Note: Answer: For every node: all values in left subtree are less, all values in right subtree are greater. Operations average $O(\log n)$ but $O(n)$ worst case if unbalanced. Self-balancing variants: AVL, Red-Black trees.

Q25. Difference between BFS and DFS?

Note: Answer: BFS (Breadth-First Search) explores level by level, uses Queue, finds shortest path in unweighted graphs. DFS (Depth-First Search) goes deep first, uses Stack (or recursion), uses less memory for wide trees, useful for topological sort and cycle detection.

SQL Questions

Q26. What is the difference between INNER JOIN and LEFT JOIN?

Note: Answer: INNER JOIN returns only rows with matches in both tables. LEFT JOIN returns all rows from the left table plus matching rows from the right table (NULL if no match). RIGHT JOIN is opposite. FULL OUTER JOIN returns all rows from both tables.

Q27. What is normalization?

Note: Answer: Process of organizing data to reduce redundancy and improve integrity. Common forms: 1NF (atomic values, no repeating groups), 2NF (1NF + no partial dependencies), 3NF (2NF + no transitive dependencies), BCNF (stricter version of 3NF).

Q28. What is the difference between DELETE, TRUNCATE, and DROP?

Note: Answer: DELETE removes rows (can use WHERE, can be rolled back, slower). TRUNCATE removes all rows (faster, resets identity, cannot use WHERE). DROP removes entire table including structure (cannot be rolled back in most DBs).

Q29. What are ACID properties?

Note: Answer: Atomicity (all or nothing - transaction completes fully or not at all), Consistency (DB remains in valid state), Isolation (concurrent transactions don't interfere), Durability (committed changes persist even after crash).

Q30. What is an index in a database?

Note: Answer: A data structure that improves query speed at the cost of additional storage and slower writes. Like an index in a book. Common types: B-tree (most common), Hash, Bitmap. Create on columns used frequently in WHERE, JOIN, ORDER BY.

Operating Systems

Q31. Process vs Thread?

Note: Answer: A process is an independent program execution with its own memory space. A thread is a lightweight unit within a process sharing the process's memory. Threads communicate easily but are less isolated. Multiple threads in one process; multiple processes need IPC.

Q32. What is a deadlock?

Note: Answer: Situation where two or more processes are stuck waiting for resources held by each other. Four conditions (Coffman conditions): Mutual exclusion, Hold and wait, No preemption, Circular wait. Prevention requires breaking at least one condition.

Q33. What is virtual memory?

Note: Answer: A memory management technique that gives processes the illusion of having more memory than physically available. Uses disk space (page file/swap) to extend RAM. Pages are swapped between RAM and disk as needed.

Networking

Q34. TCP vs UDP?

Note: Answer: TCP is connection-oriented, reliable, ordered, slower (used for HTTP, FTP, email). UDP is connectionless, unreliable, faster (used for video streaming, DNS, gaming). TCP guarantees delivery; UDP doesn't.

Q35. What happens when you type a URL in the browser?

Note: Answer: (1) Browser checks cache. (2) DNS lookup converts domain to IP. (3) TCP connection established (3-way handshake). (4) If HTTPS, TLS handshake. (5) Browser sends HTTP request. (6) Server processes and sends response. (7) Browser parses HTML/CSS/JS and renders the page.

Q36. Difference between HTTP and HTTPS?

Note: Answer: HTTP is unencrypted, uses port 80. HTTPS is HTTP over TLS/SSL, encrypted, uses port 443. HTTPS requires SSL certificate, prevents man-in-the-middle attacks, and is now standard for all websites.

Q37. What is REST API?

Note: Answer: REST (Representational State Transfer) is an architectural style for web services. Uses HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources. Stateless, client-server, cacheable, uniform interface.

O38. GET vs POST?

Note: Answer: GET retrieves data (idempotent, cacheable, params in URL, limited size). POST submits data (not idempotent, not cacheable, params in body, no size limit). Use GET for fetching, POST for creating/modifying.

32. HR & Behavioral Questions

HR rounds assess your communication, attitude, cultural fit, and career goals. Prepare structured, honest answers.

The STAR Method

For behavioral questions, structure your answer using STAR:

- **S - Situation:** Set the context. Describe the scenario.
- **T - Task:** Explain your responsibility or goal.
- **A - Action:** Detail the specific steps YOU took.
- **R - Result:** Share the outcome and what you learned.

Common HR Questions

Q1. Tell me about yourself.

Note: Strategy: Use the Present-Past-Future formula. (1) Current: your education/recent role and key skills. (2) Past: relevant experience, projects, achievements. (3) Future: why you're excited about this role. Keep it under 2 minutes. Avoid personal details unrelated to the job.

Q2. What are your strengths?

Note: Strategy: Pick 2-3 strengths relevant to software engineering: problem-solving, quick learning, teamwork, attention to detail, coding skills. For each, give a brief example. Example: 'I'm a quick learner — during my internship, I picked up React in 2 weeks and shipped a feature.'

Q3. What are your weaknesses?

Note: Strategy: Be honest but strategic. Pick a real weakness, then show how you're addressing it. Avoid cliches like 'I'm a perfectionist.' Example: 'I used to hesitate to ask for help. I've learned that asking questions early prevents wasted effort, so I now actively reach out to teammates when stuck.'

Q4. Why do you want to join our company?

Note: Strategy: Research the company beforehand. Mention specific things: their technology stack, products, culture, mission, or recent news. Connect them to your goals. Avoid generic answers like 'good company' or 'high salary.'

Q5. Where do you see yourself in 5 years?

Note: Strategy: Show ambition aligned with the company. Mention growth from associate engineer to senior/lead engineer, mastering new technologies, possibly mentoring juniors. Avoid mentioning leaving the company or unrelated career changes.

Q6. Why should we hire you?

Note: Strategy: Match your skills to job requirements. Mention 2-3 strong points: technical skills, attitude, learning ability, and fit with company culture. Be confident but not arrogant.

Q7. Why did you choose Computer Science / Software Engineering?

Note: Strategy: Share a genuine origin story — maybe a project, a moment of curiosity, or a problem you solved. Show passion for technology and problem-solving.

Q8. Tell me about a challenging project.

Note: Strategy: Use STAR method. Describe the project, the challenge (technical or interpersonal), what you specifically did, and the outcome. Highlight skills used: debugging, learning new tech, collaboration.

Q9. How do you handle pressure / tight deadlines?

Note: Strategy: Show your process: prioritize tasks, break work into chunks, communicate with team, focus on deliverables. Give a specific example. Mention you remain calm and methodical under pressure.

Q10. Describe a time you worked in a team.

Note: Strategy: Use STAR. Highlight your role, communication skills, handling conflicts, and team success. Mention tools used (Git, Slack, JIRA). Show you can collaborate and respect different viewpoints.

Q11. How do you handle conflict with a teammate?

Note: Strategy: Show emotional intelligence. Steps: listen first, understand their perspective, communicate respectfully, focus on the problem (not the person), seek common ground, and involve a manager only if needed.

Q12. What is your biggest achievement?

Note: Strategy: Pick a relevant technical or academic accomplishment: a complex project, hackathon win, research paper, internship success, or significant contribution. Quantify if possible (e.g., 'reduced load time by 40%').

Q13. How do you learn new technologies?

Note: Strategy: Show curiosity and systematic learning: official docs, online courses (Coursera, Udemy), tutorials, building projects, reading source code, following blogs/YouTube channels, participating in communities.

Q14. Do you prefer working alone or in a team?

Note: Strategy: Honest answer: both. Explain when each is appropriate. Solo work for deep focus on individual tasks; teamwork for big projects, brainstorming, learning. Show flexibility.

Q15. Why are you leaving your current job? (if applicable)

Note: Strategy: Stay positive. Never speak negatively about current/previous employer. Focus on growth: 'Looking for new challenges,' 'Want to work with new technologies,' 'Seeking growth opportunities aligned with my career goals.'

Q16. What are your salary expectations?

Note: Strategy: Research the industry standard for the role and location. Give a range, not an exact figure. Example: 'Based on my research and skills, I'm looking for [X-Y] range, but I'm open to discussion based on the overall package.'

Q17. Do you have any questions for us?

Note: Strategy: ALWAYS have questions ready. Good ones: (1) What does a typical day look like? (2) What are the team's biggest challenges? (3) What's the tech stack and learning opportunities? (4) How is performance evaluated? Avoid asking about salary/leave upfront.

Q18. Describe a time you failed.

Note: Strategy: Show maturity. Pick a real failure, explain what went wrong, what you learned, and how you've applied that lesson. Focus on the lesson, not the failure itself.

Q19. How do you prioritize tasks?

Note: Strategy: Mention frameworks: Eisenhower matrix (urgent vs important), MoSCoW (Must, Should, Could, Won't), or simple prioritization based on deadlines and impact. Communicate with stakeholders when priorities conflict.

Q20. Are you willing to relocate / work in shifts?

Note: Strategy: Answer honestly based on your situation. If yes, express enthusiasm. If no, explain politely. Don't lie — it will create problems later.

33. Tips & Best Practices

Final advice to maximize your chances of cracking your associate software engineer interview.

Resume Tips

- Keep it to 1 page (2 only if you have significant experience).
- Use a clean, professional template — avoid fancy designs.
- Mention key projects with technologies used and your contribution.
- Quantify achievements: 'Improved performance by 30%' is better than 'Improved performance.'
- List technical skills: programming languages, frameworks, tools, databases.
- Include relevant coursework, certifications, and online courses.
- Add GitHub link with active repositories — interviewers DO check.
- Include LinkedIn profile and ensure it's up-to-date.
- Use action verbs: developed, designed, implemented, optimized, led.
- Tailor your resume to each job role — match keywords from the job description.
- Proofread thoroughly — typos are immediate red flags.

Before the Interview

- Research the company — products, tech stack, culture, recent news.
- Understand the job description thoroughly. Identify required skills.
- Prepare your own questions for the interviewer.
- Practice coding problems on LeetCode, HackerRank, GeeksforGeeks.
- Mock interviews with friends or platforms like Pramp, Interviewing.io.
- Review your resume — be ready to discuss every project and skill listed.
- Prepare a 60-second 'tell me about yourself' pitch.
- Plan logistics: location, attire (business casual for tech), required documents.
- For virtual interviews: test camera, mic, internet, and lighting beforehand.
- Get a good night's sleep before the interview.

During the Coding Interview

- **Understand the problem fully** — ask clarifying questions. Don't assume.
- **Discuss examples and edge cases** — empty input, negative numbers, duplicates, large inputs.
- **Think out loud** — share your thought process. Silent coding looks like you're stuck.
- **Start with a brute force solution** if optimal isn't immediately clear, then optimize.
- **Discuss trade-offs** — time complexity vs space complexity.
- **Write clean, readable code** — meaningful variable names, proper indentation.
- **Test your code** manually with examples. Walk through line by line.
- **Handle edge cases** — null/empty inputs, boundary values.
- **Don't panic if stuck** — explain what you've tried, ask for hints if allowed.
- **Discuss the complexity** at the end — both time and space.

Communication Tips

- Speak clearly and confidently. Maintain good eye contact (or look at camera for virtual).
- Listen carefully to the interviewer — don't interrupt.
- Be honest about what you don't know. Saying 'I don't know but I'd approach it like this...' is better than guessing.
- Use technical terms correctly — but explain in simple terms if asked.
- Smile occasionally — it makes you appear approachable and confident.
- Body language matters: sit up straight, don't fidget, nod to show you're listening.

Common Mistakes to Avoid

- **Jumping into code without understanding the problem** — leads to wrong solutions.
- **Not asking clarifying questions** — assumptions cause errors.
- **Silent coding** — interviewer can't help if they don't know your thinking.
- **Memorizing solutions without understanding** — interviewers ask follow-ups.
- **Speaking negatively about previous companies/colleagues** — major red flag.
- **Lying on resume** — easily caught and instant disqualification.
- **Being arrogant or dismissive** — being humble and coachable is valued.
- **Not preparing questions** — shows lack of interest.
- **Poor time management** — practice solving problems within 30-45 minutes.
- **Forgetting to test code** — bugs in front of the interviewer hurt.
- **Ignoring edge cases** — empty inputs, overflow, null values.

Recommended Study Plan (4-6 Weeks)

Week	Focus Areas
Week 1	Python fundamentals: syntax, data types, control flow, functions, OOP
Week 2	Data Structures: lists, dicts, stacks, queues, linked lists, trees
Week 3	Algorithms: sorting, searching, recursion, basic DP, time complexity
Week 4	DBMS, SQL queries, OS basics, networking fundamentals
Week 5	Mock interviews, solve 50+ coding problems, system design basics
Week 6	HR prep, behavioral questions, company-specific research, final revision

Best Resources for Practice

- **Coding Practice:** LeetCode, HackerRank, CodeChef, GeeksforGeeks, Codeforces
- **Python:** Official docs (python.org), Real Python, Corey Schafer (YouTube)
- **DSA:** 'Cracking the Coding Interview' book, NeetCode (YouTube), Striver's SDE Sheet
- **System Design:** ByteByteGo, System Design Primer (GitHub)
- **Mock Interviews:** Pramp, Interviewing.io, peer practice
- **SQL:** SQLZoo, HackerRank SQL, LeetCode Database section

- **Aptitude:** IndiaBix, Sawaal, PrepInsta (especially for Indian campus placements)

Day-of-Interview Checklist

- Arrive 15 minutes early (or log in 10 minutes early for virtual)
- Carry printed resume copies, ID proof, and any required documents
- Dress appropriately (business casual is safe for tech interviews)
- Bring a notepad and pen for taking notes
- Have water nearby — dry mouth from nerves is common
- Turn off phone notifications
- Take deep breaths before starting — calm mind = clear thinking
- Smile, greet the interviewer warmly, and thank them at the end

After the Interview

- Send a thank-you email within 24 hours, mentioning specific discussion points
- Reflect on what went well and what could improve for next time
- Write down questions you couldn't answer — review them for future interviews
- Don't dwell on mistakes — every interview is a learning experience
- Continue applying and practicing — don't put all hopes on one interview

Final Words

Consistency beats intensity. Studying 2 hours daily for 30 days is better than 10 hours for 3 days. Build a habit, stay patient, and trust the process.

Confidence comes from preparation. The more problems you solve and concepts you understand, the more confident you'll be in the interview room.

Rejection is not a reflection of your worth. Even great engineers face rejections. Learn from each interview and keep going.

Build a portfolio. A few well-documented projects on GitHub speak louder than many bullet points on a resume.

Network actively. Connect with engineers on LinkedIn, attend tech meetups, contribute to open source. Many jobs come through referrals.

All the best for your interview! You've got this.